

An interpretable cross-domain generalization method for mental health NLP: stable feature selecting and residual fusion

student ID:1786622

Xingjian Sun

Structured abstract

Objective:

This research hope to propose a cross-domain generalization method for health NLP models, which does not require additional data labeling, and can also retain certain interpretability.

Materials and Methods:

This research use two public datasets with large differences to simulate cross-domain scenarios in the health field, among which GoEmotions represents shorter, more colloquial social media text, and IMDb represents longer, more coherent review text. This research extract some hand-designed features from the text, and screen out features that can still work stably after cross-domain through statistical tests, on this basis, design a residual fusion framework, so that these stable features can correct the mistakes left by the model during the pre-training stage.

Results:

The cross-domain stability of style feature and structure features is significantly higher than that of lexical features. After fusing the small model trained with stable features with the original large model, the new model is slightly better than BERT in the original training scenario, and the overall performance in the new scenario is also basically stable.

Discussion:

The value of stable features is not only whether they can provide additional information, but more importantly, they can help the model correct those predictions that “seem very certain but are actually wrong”, and this kind of mistake is especially dangerous in health scenarios.

Conclusion:

This research provide a cross-domain method more suitable for medical and health scenarios: it does not need to spend a lot of time and resources to re-label for new scenarios, and can also provide a clearer explanatory basis for model judgment.

Introduction

Mental health screening has long faced two practical problems: first, professional assessment usually requires a lot of clinical personnel to participate, the labor cost is high; second, clinical resources are insufficient in many scenarios, leading to the difficulty of screening to cover more people in a timely manner.

Now, the screening of mental health related information on the market usually rely on doctors or counselors to carry out, which have high cost and limited coverage. Therefore NLP is regarded as a automatic screening method that can use clinical records, forum posts and social media content[1,

2, 9, 16, 17]. But when really apply NLP models to mental health related scenarios, the text that the model contact during training and the text it face in actual application are often not consistent in length, style and expression way[18, 19]. Social media text are usually shorter and more oral, while clinical record text are generally longer and more formal[8]. In this case, when the model switch to new scenes, its performance will drop, even make mistake. And the mental health related field often need to do transfer across hospitals, across platforms, or across text. Every new scene’s newly collected and labeled data are also restricted by privacy and compliance[8]. Existing methods are all trying to solve this problem, but each have their own shortcomings: vocabulary meaning screening or manual feature method are relatively stable, but not good at understanding more complex semantics[5]; BERT pre-trained model[4] perform well on single dataset, but when cross-domain, it will have the problem that it judge wrong but still very confident[7], which is especially dangerous in health scenarios. Therefore, the core purpose of this research is, while keep the strong ability of the pre-trained model[10–15], to introduce a more stable, and also easier to explain cross-domain feature. Not all text feature will fluctuate greatly with different platforms, features like writing style, sentence structure and length pattern, are often more stable than specific words. If we extract these stable features separately, and use them to assist or correct the pre-trained model, it is possible to improve the cross-domain generalization ability. Around this idea, this research focus on three questions: which type of text-level features are the most stable between different domains, whether these stable features can really improve cross-domain classification performance, and whether they are “supplementing new information” or “correcting the mistakes that the model would have made originally”.

Methods

Ethics Statement

All the data used in this research are publicly available academic datasets, and are only used for non-commercial academic research. During the research process, we will not redistribute any original raw text data, nor will we redistribute any derived data generated during this project. We also do not attempt to identify or extract any personal identity information from the data. This research uses health-related text data, which is inherently sensitive. To minimize the related privacy risks, we only use aggregated statistical features extracted from the original text and model output results for analysis, avoiding the use of individual-level sensitive information for any other purposes.

Data selection

This research use two public corpora as proxy domains. GoEmotions contains about 58,000 short Reddit comments[3], the original labels cover 27 fine-grained emotions, the text is short and colloquial, close to daily social platform expression. IMDb contains 50,000 long movie reviews[6], the labels are only positive and negative, the text is longer and more coherent as a whole, and also closer to complete written reviews. This research choose these two datasets, not to take them directly as real health corpus, but to regard them as proxy domains with significant differences in style, length and expression habits, to test whether the method is still effective under obvious distribution changes. To prove that the cross-domain experiment is indeed testing whether the model can maintain performance in a completely new scenario, we first need to prove that the difference

between these two datasets used to simulate different scenarios is large enough. As shown in Figure 1, these two datasets have obvious differences in data scales and text length; IMDb text is longer as a whole, the average number of words is more than 230, while GoEmotions text is shorter and more fragmented, the average number of words is about 13. More detailed statistics are summarized in Table 1.

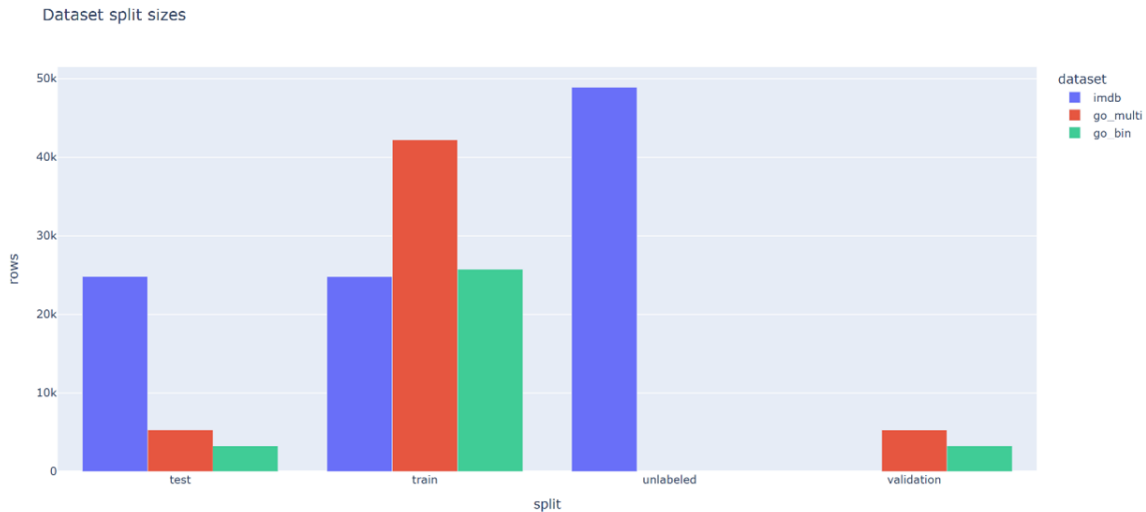


Figure 1. Dataset split sizes

Table 1. dataset__overview

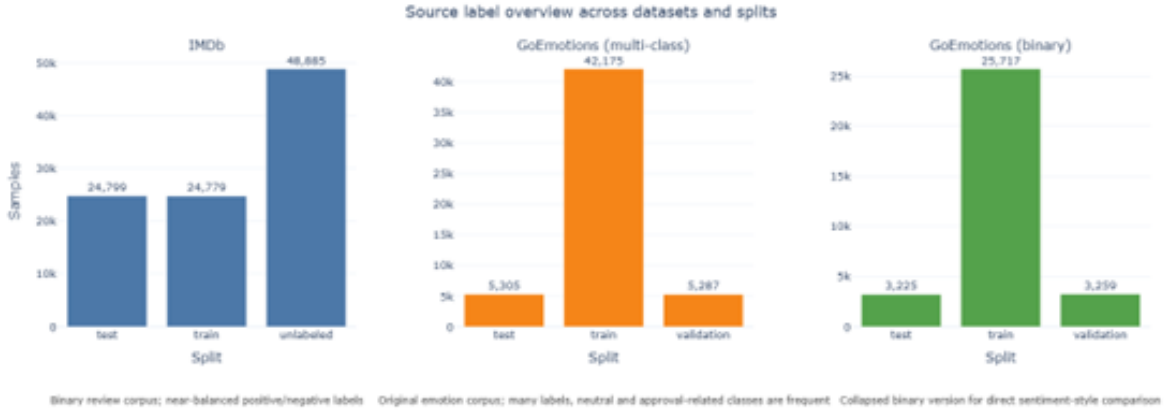
Dataset	Text Source	Size	Labels	Avg. Words
GoEmotions	Reddit comments	58,000	27 fine-grained emotions	13
IMDb	Movie reviews	50,000	Positive and negative	>230

Label Alignment

The original label systems of the two datasets are different, so before carrying out the cross-domain experiment, this research first do label alignment. Specifically, this research map the 27 emotion labels of GoEmotions into three coarser sentiment groups: positive, negative, neutral/other. The purpose of this is to ensure the label comparability between different datasets, to avoid the subsequent comparison being interfered by the inconsistency of label standards. This research use pre-defined fixed mapping rules, not to adjust the rules temporarily according to the experimental results, this can avoid the problem of “changing labels to make the results look better”, and also help ensure the process is reproducible. The mapping rules are listed in Table 2, and the aligned label distribution is shown in Figure 3.

Table 2. label_mapping

Target Group	Original Labels
Positive	admiration, amusement, approval, caring, desire, excitement, gratitude, joy, love, optimism, pride, relief
Negative	anger, annoyance, disappointment, disapproval, disgust, embarrassment, fear, grief, nervousness, remorse, sadness
Neutral / Other	neutral, curiosity, realization, surprise

**Figure 2.** Aligned label distribution visualization

Stable Feature Selection

The core of this research is not “stack as many features as possible”, but “find out the features that are still reliable after cross-domain”. For this, this research extract 42 candidate features from five types of features: lexical features, sentiment features, style features, structure features and semantic features. These candidate features cover multiple levels from specific words to expression methods, to avoid simplifying the cross-domain problem into a single-dimensional difference. When screening these features, this research follow two principles: first, the features themselves should have clear linguistic meaning, easy to explain and audit; second, the feature set should try to cover different text levels that are most likely to change or remain stable when cross-domain.

Next, this research perform stability screening on these features. The “stable” here does not require a feature to be exactly the same in the two domains, but requires that it cannot be too different after changing domains, otherwise it is likely to fail when cross-domain. For this, this research use a variety of statistical tools at the same time, to judge whether a feature is suitable for cross-domain use from three aspects: significance, degree of difference and feature redundancy. The statistical methods used in this research are: KS test, FDR correction, JS divergence and VIF test. The related tools and their function are show in Table 3.

Table 3. related tools and function

Tool	Function
KS test	Significance testing for cross-domain distribution difference
FDR correction	Multiple-comparison correction during feature screening
JS divergence	Degree of cross-domain distribution difference
VIF test	Feature redundancy / multicollinearity checking

We set five comparison model L1-L5, from the bag-of-words model that only look at surface vocabulary, to BERT baseline, the model that only use stable features, the residual fusion model of stable features and BERT, and the model that add all 42 candidate features. The purpose of these model is not just to compare the score, but to verify whether the stable feature themselves are useful, whether the fusion method is effective, and whether the previous stability screening is really important.

Result

Results of Feature Stability (Research Question 1)

The final result is quite significant: the average JS divergence of style features is the lowest, at 0.04, and the pass rate is the highest, at 90%; structure features is next, average JS divergence is 0.06, pass rate is 85%; in contrast, the JS divergence of lexical features is the highest, at 0.21, pass rate is the lowest, only 30%. This show that specific vocabulary is easier to change and fail with the corpus, while style and structure are closer to the unchanged expression pattern, so they are more suitable to be used as stable features in cross-domain scenes. This result is also shown more intuitively in Figure 4.

Table 4. Summary of feature stability statistical results

Feature Group	Average JS divergence	Pass rate
Style features	0.04	90%
Structure features	0.06	85%
Lexical features	0.21	30%

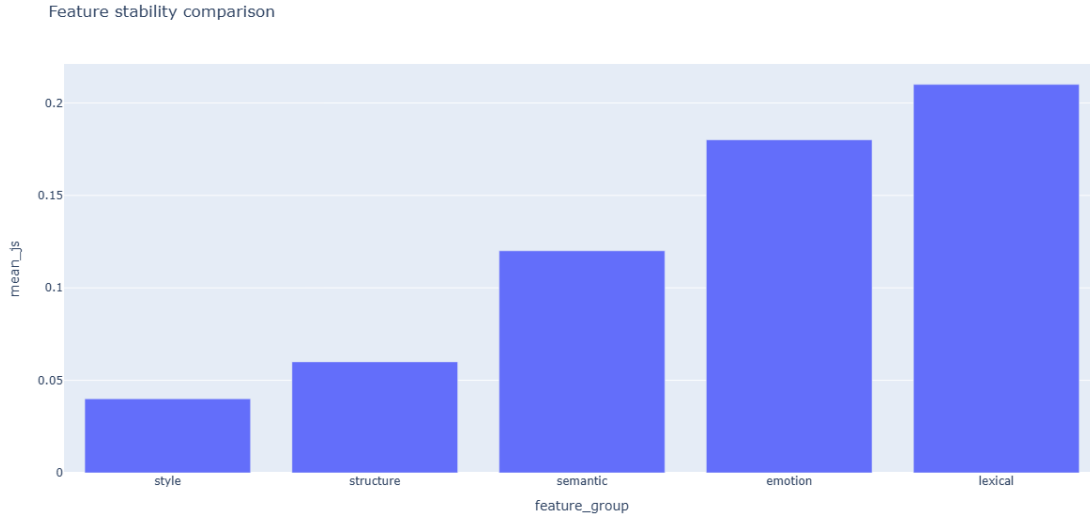


Figure 3. Feature stability comparison visualization

Model Performance Results (Research Question 2)

After confirming which features are more stable, this research further examine whether these features can really improve cross-domain classification performance. As shown in Table 5, the F1 score of the BERT baseline model L2 on the source domain validation set is 87.2%, after adding stable features and using residual fusion, L4 increased the F1 to 89.5%, which indicates that stable features can indeed improve the overall performance of the model in the source domain. On the cross-domain target domain test set, the F1 score of the BERT baseline is 0.737, L5 which uses all features achieved almost the same 0.737, while the residual fusion model L4 also achieved similar performance. This shows that in cross-domain scenarios, stable features can at least provide additional error correction ability for the model without significantly losing the main task performance. Figure 5 shows this result more intuitively: L4 performs better than other models, while L5 does not continue to improve as features increase.

Table 5. Model performance summary table

Model				F1 / score	Note
L2	BERT	baseline	(source domain)	87.2%	Source domain validation set
L4	residual fusion	(source domain)		89.5%	Source domain validation set
L2	BERT	baseline	(target domain)	0.737	Cross-domain target domain test set
L5	all features	(target domain)		0.737	Cross-domain target domain test set
L4	residual fusion	(target domain)		Similar performance	Cross-domain target domain test set

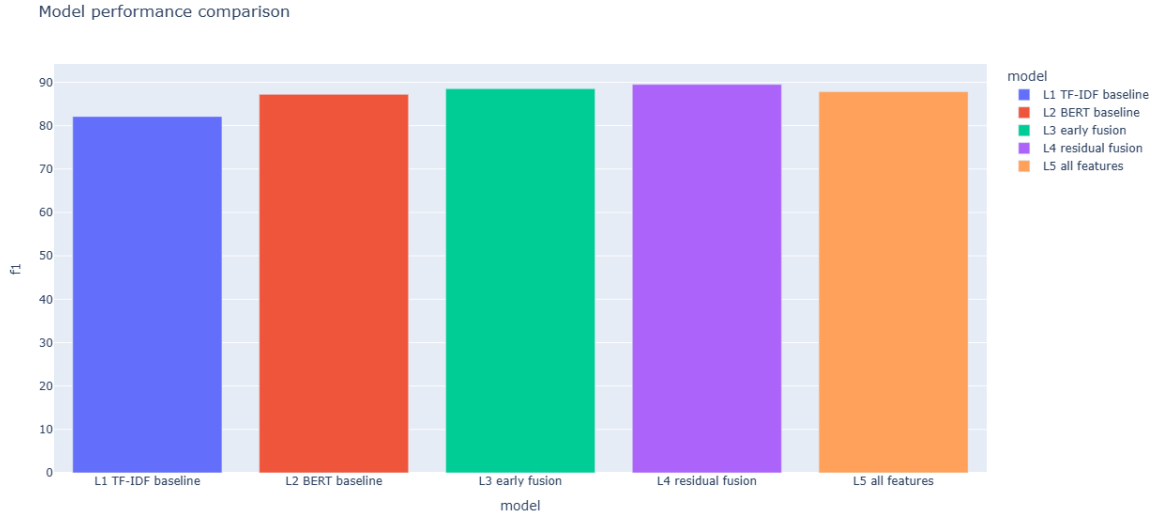


Figure 4. Model performance comparison visualization

Error Correction Mechanism (Research Question 3)

In addition to looking at whether the score has improved, this research further analyze how stable features work exactly. The overall performance metric can only tell “whether the model has improved”, but cannot explain “how the model has improved”, so this research further analyze the error correction rate ECR, to judge whether stable features are adding new information, or correcting the mistakes that the text model would have made originally. We can understand ECR as a very intuitive metric: among the samples that the original text model made mistakes, how many of them are successfully corrected after fusion. As shown in Table 6, the overall ECR is 8.2%, and this indicator will rise as the text model’s confidence increases; especially in the highest confidence interval, that is, greater than 0.9, the ECR reaches 18.7%. This means that the more the model “thinks it is right”, the more likely the stable features will find that it is actually wrong, and correct the mistake back. Figure 6 shows this trend more clearly.

Table 6. ECR statistical results

Setting	ECR
Overall	8.2%
Highest confidence interval (> 0.9)	18.7%

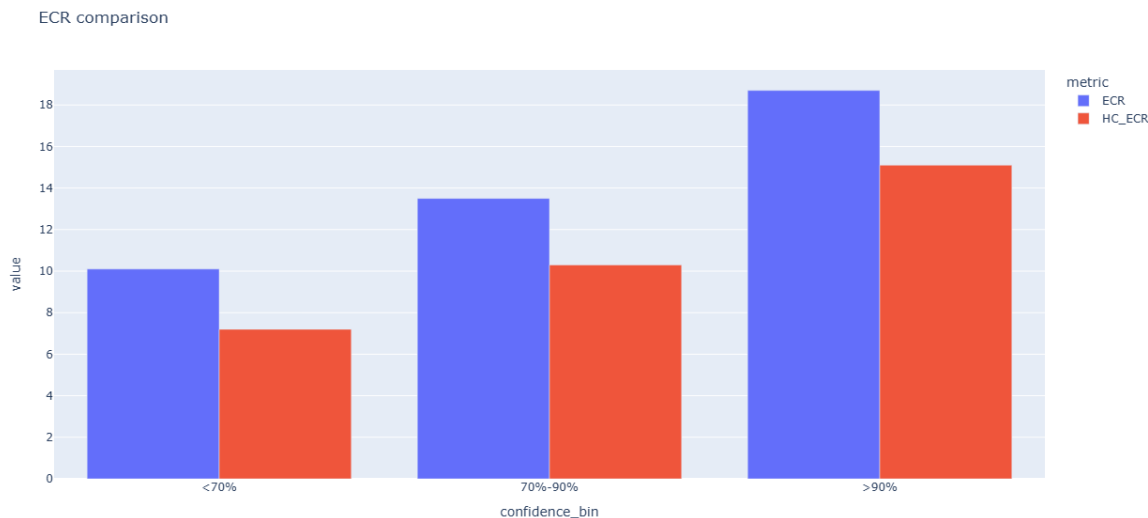


Figure 5. ECR trend chart with confidence

Discussion

The core findings of this research at the methodological level can be summarized into three points. The main findings of this research can be summarize into three points: First, compared with what specific words are used, the way text is expressed is often more stable when cross-domain, so what is really more useful is often not “what is written”, but “how it is written”; Second, more features are not always better, the key is to first screen out those features that can remain stable after cross-domain, because many features that are only useful in the old scenario will becomes noise after switching to the new scenario; Third, the role of stable features is not to replace pre-trained models like BERT, but to do error correction after it, specifically to find and correct those result that the model “is very confident but actually judge wrong”.

Significance at the application level

When applied to health NLP scenarios, the significance of this method is also relatively direct: it do not need to re-do a lot of labeling for new scenarios, which is more suitable for the reality of expensive data and difficult labeling in the medical field; At the same time, what it mainly use is the aggregated stable features, rather than directly transferring the original sensitive text, so it is also easier to meet privacy and compliance requirements. In addition, real medical system often need to constantly change departments, platforms and text sources, so cross-domain stability is not an extra bonus, but a prerequisite for whether the model can be really used. From the perspective of actual deployment, this method is more suitable to be placed after the original model, as a second-layer checking mechanism: first, the original model complete the conventional judgment, then the stable feature module do a supplementary check, focusing on intercepting those dangerous predictions that seem very certain but may actually be wrong.

limitations

Of course, this research also have obvious limitations. First, since we use general text rather than real clinical corpus, the current conclusion is more suitable to be understood as method feasibility verification, rather than a direct proof of the effect of specific health tasks. Second, this research only test a limited number of hand-designed features, so the current results can better explain that “the stable feature idea itself is effective”, but cannot explain that we have found the optimal set of stable features. Third, this research currently focus on sentiment classification tasks, so we cannot directly extrapolate the conclusion to more complex multi-classification, risk prediction or sequence labeling tasks in health NLP.

Future Work

Future work will focus on these limitations. Specifically, this research plan to apply this framework to real health text scenarios, especially mental health and clinical text analysis tasks. At the same time, this research also hope to explore more automatically learned stable features, to reduce the cost of manual design, and test whether there is a better stable feature space. In addition, the future should further evaluate the performance of this method in calibration, error interpretability and human-machine collaboration processes, to verify its usability in real health scenarios[20].

Conclusion

Overall, the method proposed in this research is not to replace the original model, but to add a cross-domain error correction mechanism after it. On the one hand, it retains the ability of large models to understand complex text, on the other hand, it uses stable features to improve the robustness of the model in new scenarios. Therefore, it is closer to the kind of method that health NLP scenarios really need: not only high scores in the original scenarios, but also can be used, explained, and easier to deploy after changing scenarios.

Reference

- [1] Benton, A., Mitchell, M., & Hovy, D. (2017). Multitask learning for mental health conditions with limited social media data. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 1, Long Papers* (pp. 152–162). Association for Computational Linguistics.
- [2] Coppersmith, G., Dredze, M., & Harman, C. (2014). Quantifying mental health signals in Twitter. In *Proceedings of the Workshop on Computational Linguistics and Clinical Psychology: From Linguistic Signal to Clinical Reality* (pp. 51–60). Association for Computational Linguistics.
- [3] Demszky, D., Movshovitz-Attias, D., Ko, J., Cowen, A. S., Nemade, G., & Ravi, S. (2020). GoEmotions: A dataset of fine-grained emotions. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 4040–4054). Association for Computational Linguistics.

- [4] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (pp. 4171–4186). Association for Computational Linguistics.
- [5] Ramponi, A., & Plank, B. (2020). Neural unsupervised domain adaptation in NLP—A survey. In *Proceedings of the 28th International Conference on Computational Linguistics* (pp. 6838–6855). International Committee on Computational Linguistics. <https://doi.org/10.18653/v1/2020.coling-main.603>
- [6] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning word vectors for sentiment analysis. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies* (pp. 142–150). Association for Computational Linguistics.
- [7] Nguyen, A., Yosinski, J., & Clune, J. (2015). Deep neural networks are easily fooled: High confidence predictions for unrecognizable images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 427–436). IEEE.
- [8] Tamang, S., Humbert-Droz, M., Gianfrancesco, M., Izadi, Z., Schmajuk, G., & Yazdany, J. (2023). Practical considerations for developing clinical natural language processing systems for population health management and measurement. *JMIR Medical Informatics*, 11, e37805. <https://doi.org/10.2196/37805>
- [9] Turcan, E., & McKeown, K. (2019). Dreaddit: A Reddit dataset for stress analysis in social media. In *Proceedings of the Tenth International Workshop on Health Text Mining and Information Analysis (LOUHI 2019)* (pp. 97–107). Association for Computational Linguistics.
- [10] Howard, J., & Ruder, S. (2018). Universal language model fine-tuning for text classification. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (pp. 328–339). Association for Computational Linguistics.
- [11] Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long Papers)* (pp. 2227–2237). Association for Computational Linguistics.
- [12] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692.
- [13] Yang, Z., Dai, Z., Yang, Y., Carbonell, J., Salakhutdinov, R., & Le, Q. V. (2019). XLNet: Generalized autoregressive pretraining for language understanding. In *Advances in Neural Information Processing Systems*, 32.
- [14] Lan, Z., Chen, M., Goodman, S., Gimpel, K., Sharma, P., & Soricut, R. (2019). ALBERT: A Lite BERT for self-supervised learning of language representations. *CoRR*, abs/1909.11942.
- [15] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter. *arXiv preprint arXiv:1910.01108*.

- [16] Zhang, T., Schoene, A. M., Ji, S., & Ananiadou, S. (2022). Natural language processing applied to mental illness detection: A narrative review. *npj Digital Medicine*, 5(1), 46. <https://doi.org/10.1038/s41746-022-00589-7>
- [17] Malgaroli, M., Hull, T. D., Zech, J. M., & Althoff, T. (2023). Natural language processing for mental health interventions: A systematic review and research framework. *Translational Psychiatry*, 13(1), 309. <https://doi.org/10.1038/s41398-023-02592-2>
- [18] Aragón, M. E., López-Monroy, A. P., Montes-y-Gómez, M., & Losada, D. E. (2025). Adapting language models for mental health analysis on social media. *Artificial Intelligence in Medicine*, 168, 103217. <https://doi.org/10.1016/j.artmed.2025.103217>
- [19] Bhatt, K., Singh, A. K., Pandey, P., Vishnoi, A., & Narooka, P. (2026). Cross platform social media analysis for mental health detection. *Discover Mental Health*, 6(1), 33. <https://doi.org/10.1007/s44192-026-00368-w>
- [20] Ulmer, D., Frellsen, J., & Hardmeier, C. (2022). Exploring predictive uncertainty and calibration in NLP: A study on the impact of method & data scarcity. In *Findings of the Association for Computational Linguistics: EMNLP 2022* (pp. 2707–2735). Association for Computational Linguistics. <https://doi.org/10.18653/v1/2022.findings-emnlp.198>

Appendix

Project Code

```

1 \appendix
2
3 \section*{Appendix}
4 \addcontentsline{toc}{section}{Appendix}
5
6 \subsection*{prep.ipynb}
7 \begin{lstlisting}[language=Python, caption={prep.ipynb}]
8 # Run this once if needed
9 # %pip install -q datasets pandas requests pyarrow numpy tqdm
10
11 from pathlib import Path
12 import json
13
14 root = Path.cwd()
15 data_dir = root / "data"
16 out_dir = root / "outputs"
17 prep_dir = out_dir / "data_prep"
18
19 go_raw = data_dir / "go_raw"
20 imdb_raw = data_dir / "imdb_raw"
21
22 for p in [data_dir, out_dir, prep_dir, go_raw, imdb_raw]:
23     p.mkdir(parents=True, exist_ok=True)
24
25 cfg = {
26     "seed": 42,
27     "min_words": 3,
28     "length_bins": [0, 32, 64, 128, 256, 10**9],
29     "length_labels": ["0-32", "33-64", "65-128", "129-256", "256+"],
30     "priority": [
31         "admiration", "amusement", "approval", "caring", "desire", "excitement",
32         "gratitude", "joy", "love", "optimism", "pride", "relief",
33         "anger", "annoyance", "disappointment", "disapproval", "disgust",
34         "embarrassment", "fear", "grief", "nervousness", "remorse", "sadness",

```

```

33         "curiosity", "realization", "surprise", "neutral"
34     ],
35     "positive": [
36         "admiration", "amusement", "approval", "caring", "desire", "excitement",
37         "gratitude", "joy", "love", "optimism", "pride", "relief"
38     ],
39     "negative": [
40         "anger", "annoyance", "disappointment", "disapproval", "disgust",
41         "embarrassment", "fear", "grief", "nervousness", "remorse", "sadness"
42     ],
43     "neutral_other": ["neutral", "curiosity", "realization", "surprise"]
44 }
45
46 paths = {
47     "root": str(root),
48     "data_dir": str(data_dir),
49     "out_dir": str(out_dir),
50     "prep_dir": str(prepare_dir),
51     "go_raw": str(go_raw),
52     "imdb_raw": str(imdb_raw),
53     "clean_imdb_binary": str(data_dir / "clean_imdb_binary.csv"),
54     "clean_goemotions_multiclass": str(data_dir / "clean_goemotions_multiclass.csv"),
55     "clean_goemotions_binary": str(data_dir / "clean_goemotions_binary.csv"),
56     "corpus_profile_report": str(prepare_dir / "corpus_profile_report.csv"),
57     "dataset_validation_report": str(prepare_dir / "dataset_validation_report.csv"),
58     "readiness_check": str(prepare_dir / "readiness_check.csv"),
59     "go_zero_label": str(prepare_dir / "go_zero_label.csv"),
60     "process_log": str(prepare_dir / "process_log.txt"),
61     "config": str(prepare_dir / "config.json")
62 }
63
64 with open(prepare_dir / "config.json", "w", encoding="utf-8") as f:
65     json.dump({"paths": paths, "cfg": cfg}, f, ensure_ascii=False, indent=2)
66
67 print("Current working directory:", root.resolve())
68 print(json.dumps(paths, ensure_ascii=False, indent=2))
69
70 import requests
71 import tarfile
72
73 imdb_url = "https://ai.stanford.edu/~amaas/data/sentiment/aclImdb_v1.tar.gz"
74 imdb_tar = imdb_raw / "aclImdb_v1.tar.gz"
75 imdb_dir = imdb_raw / "aclImdb"
76
77 if not imdb_dir.exists():
78     with requests.get(imdb_url, stream=True, timeout=120) as r:
79         r.raise_for_status()
80         with open(imdb_tar, "wb") as f:
81             for chunk in r.iter_content(chunk_size=1024 * 1024):
82                 if chunk:
83                     f.write(chunk)
84
85 if not imdb_dir.exists():
86     with tarfile.open(imdb_tar, "r:gz") as tar:
87         tar.extractall(imdb_raw)
88
89 imdb_dir
90
91 from datasets import load_dataset
92 import pandas as pd
93
94 go_ds = load_dataset("SetFit/go_emotions")
95
96 for split in go_ds.keys():
97     pd.DataFrame(go_ds[split]).to_csv(go_raw / f"{split}.csv", index=False,
98         encoding="utf-8-sig")
99
100 list(go_raw.glob("*.csv"))

```

```

98
99 import html
100 import json
101 import re
102 import numpy as np
103 import pandas as pd
104
105 url_re = re.compile(r"https?://\S+|www\.\S+", re.I)
106 space_re = re.compile(r"\s+")
107 br_re = re.compile(r"(<br\s*/?>)+", re.I)
108
109
110 def to_str(x):
111     if pd.isna(x):
112         return None
113     return str(x)
114
115
116 def clean_text(x):
117     x = to_str(x)
118     if x is None:
119         return None
120     x = html.unescape(x)
121     x = br_re.sub(" ", x)
122     x = x.replace("\r", " ").replace("\n", " ").replace("\t", " ")
123     x = url_re.sub("[URL]", x)
124     x = space_re.sub(" ", x).strip()
125     return x if x else None
126
127
128 def word_count(x):
129     return len(x.split()) if isinstance(x, str) else 0
130
131
132 def char_count(x):
133     return len(x) if isinstance(x, str) else 0
134
135
136 def avg_word_len(x):
137     toks = x.split() if isinstance(x, str) else []
138     return float(np.mean([len(t) for t in toks])) if toks else 0.0
139
140
141 def upper_ratio(x):
142     if not isinstance(x, str):
143         return 0.0
144     letters = [c for c in x if c.isalpha()]
145     if not letters:
146         return 0.0
147     return sum(1 for c in letters if c.isupper()) / len(letters)
148
149
150 def add_text_cols(df, text_col="text"):
151     df = df.copy()
152     df["text_lower"] = df[text_col].str.lower()
153     df["n_words"] = df[text_col].apply(word_count)
154     df["n_chars"] = df[text_col].apply(char_count)
155     df["avg_word_len"] = df[text_col].apply(avg_word_len)
156     df["upper_ratio"] = df[text_col].apply(upper_ratio)
157     df["exclaim_cnt"] = df[text_col].apply(lambda s: s.count("!") if isinstance(s,
158 str) else 0)
159     df["question_cnt"] = df[text_col].apply(lambda s: s.count("?") if isinstance(s,
160 str) else 0)
161     df["length_bin"] = pd.cut(
162         df["n_words"],
163         bins=cfg["length_bins"],
164         labels=cfg["length_labels"],
165         include_lowest=True,

```

```

164         right=True
165     ).astype(str)
166     return df
167
168
169 def dedup_by_split_priority(df, split_col, text_col, priority_map):
170     df = df.copy()
171     df["_split_priority"] = df[split_col].map(priority_map).fillna(9999)
172     df["_text_key"] = df[text_col].astype(str).str.strip().str.lower()
173     df = df.sort_values(by=["_split_priority", "_text_key"], ascending=[True, True])
174     df = df.drop_duplicates(subset=["_text_key"], keep="first").copy()
175     df = df.drop(columns=["_split_priority", "_text_key"])
176     return df.reset_index(drop=True)
177
178
179 def profile_table(df, name):
180     rows = []
181     for sp, part in df.groupby("split", dropna=False):
182         rows.append({
183             "dataset": name,
184             "split": sp,
185             "rows": int(len(part)),
186             "unique_text": int(part["text"].nunique()),
187             "dup_text": int(len(part) - part["text"].nunique()),
188             "mean_words": float(part["n_words"].mean()) if len(part) else np.nan,
189             "median_words": float(part["n_words"].median()) if len(part) else np.nan,
190             "p25_words": float(part["n_words"].quantile(0.25)) if len(part) else
191                 np.nan,
192             "p75_words": float(part["n_words"].quantile(0.75)) if len(part) else
193                 np.nan,
194             "min_words": float(part["n_words"].min()) if len(part) else np.nan,
195             "max_words": float(part["n_words"].max()) if len(part) else np.nan,
196             "labels": json.dumps(part["label"].value_counts(dropna=False).to_dict(),
197                                 ensure_ascii=False)
198         })
199     return pd.DataFrame(rows)
200
201 rows = []
202
203 for split in ["train", "test"]:
204     for cls, label in [("pos", 1), ("neg", 0)]:
205         folder = imdb_dir / split / cls
206         for fp in folder.glob("*.txt"):
207             rows.append({
208                 "dataset": "imdb",
209                 "split": split,
210                 "source_file": str(fp),
211                 "orig_split": split,
212                 "orig_class": cls,
213                 "label": label,
214                 "text_raw": fp.read_text(encoding="utf-8", errors="ignore")
215             })
216
217 unsup_dir = imdb_dir / "train" / "unsup"
218 for fp in unsup_dir.glob("*.txt"):
219     rows.append({
220         "dataset": "imdb",
221         "split": "unlabeled",
222         "source_file": str(fp),
223         "orig_split": "train_unsup",
224         "orig_class": "unsup",
225         "label": np.nan,
226         "text_raw": fp.read_text(encoding="utf-8", errors="ignore")
227     })
228
229 imdb = pd.DataFrame(rows)
230 imdb.insert(0, "row_id", range(1, len(imdb) + 1))
231 imdb["text"] = imdb["text_raw"].apply(clean_text)

```

```

229 imdb = imdb.dropna(subset=["text"]).copy()
230 imdb = imdb[imdb["text"].apply(word_count) >= cfg["min_words"]].copy()
231 imdb = imdb.drop_duplicates(subset=["split", "text"]).reset_index(drop=True)
232 imdb = dedup_by_split_priority(
233     imdb,
234     split_col="split",
235     text_col="text",
236     priority_map={"test": 0, "train": 1, "unlabeled": 2}
237 )
238 imdb.insert(0, "id", [f"imdb_{i}" for i in range(1, len(imdb) + 1)])
239 imdb = add_text_cols(imdb, "text")
240
241 imdb = imdb[[
242     "id", "row_id", "dataset", "split", "source_file", "orig_split", "orig_class",
243     "label", "text_raw", "text", "text_lower", "n_words", "n_chars", "avg_word_len",
244     "upper_ratio", "exclaim_cnt", "question_cnt", "length_bin"
245 ]].copy()
246
247 imdb.to_csv(data_dir / "clean_imdb_binary.csv", index=False, encoding="utf-8-sig")
248 imdb.head()
249
250 parts = []
251 for fp in sorted(go_raw.glob("*.csv")):
252     tmp = pd.read_csv(fp)
253     tmp["split"] = fp.stem
254     tmp["source_file"] = str(fp)
255     parts.append(tmp)
256
257 go = pd.concat(parts, ignore_index=True)
258
259 base_cols = {"text", "split", "source_file", "id"}
260 emotion_cols = []
261 for c in go.columns:
262     if c in base_cols:
263         continue
264     vals = pd.Series(go[c]).dropna()
265     try:
266         u = set(vals.astype(int).unique().tolist())
267         if u.issubset({0, 1}):
268             emotion_cols.append(c)
269     except Exception:
270         pass
271 emotion_cols = sorted(emotion_cols)
272
273 priority_rank = {k: i for i, k in enumerate(cfg["priority"])}
274 pos_set = set(cfg["positive"])
275 neg_set = set(cfg["negative"])
276
277
278 def active_labels(row):
279     labs = []
280     for c in emotion_cols:
281         try:
282             if int(row[c]) == 1:
283                 labs.append(c)
284         except Exception:
285             pass
286     return labs
287
288
289 def pick_main(labs):
290     if not labs:
291         return None
292     non_neutral = [x for x in labs if x != "neutral"]
293     pool = non_neutral if non_neutral else labs
294     pool = sorted(pool, key=lambda x: priority_rank.get(x, 999))
295     return pool[0] if pool else None
296

```

```

297
298 def to_bin(label):
299     if label in pos_set:
300         return 1
301     if label in neg_set:
302         return 0
303     return np.nan
304
305 go.insert(0, "row_id", range(1, len(go) + 1))
306 go["text_raw"] = go["text"].apply(to_str)
307 go["text"] = go["text_raw"].apply(clean_text)
308 go = go.dropna(subset=["text"]).copy()
309 go = go[go["text"].apply(word_count) >= cfg["min_words"]].copy()
310 go["active_labels"] = go.apply(active_labels, axis=1)
311 go["n_active"] = go["active_labels"].apply(len)
312
313 zero_label = go[go["n_active"] == 0].copy()
314 zero_label.to_csv(prepare_dir / "go_zero_label.csv", index=False, encoding="utf-8-sig")
315
316 go = go[go["n_active"] > 0].copy()
317 go["main_label"] = go["active_labels"].apply(pick_main)
318 go = go.dropna(subset=["main_label"]).copy()
319 go = go.drop_duplicates(subset=["split", "text", "main_label"]).reset_index(drop=True)
320 go = dedup_by_split_priority(
321     go,
322     split_col="split",
323     text_col="text",
324     priority_map={"test": 0, "validation": 1, "train": 2}
325 )
326 go.insert(0, "id", [f"go_{i}" for i in range(1, len(go) + 1)])
327 go = add_text_cols(go, "text")
328 go["bin_label"] = go["main_label"].apply(to_bin)
329 go["dataset"] = "go"
330
331 go_multi = go[[
332     "id", "row_id", "dataset", "split", "source_file", "text_raw", "text",
333     "text_lower",
334     "main_label", "n_active", "n_words", "n_chars", "avg_word_len",
335     "upper_ratio", "exclaim_cnt", "question_cnt", "length_bin"
336 ]].copy().rename(columns={"main_label": "label"})
337
338 go_bin = go[go["bin_label"].isin([0, 1])].copy()
339 go_bin = go_bin[[
340     "id", "row_id", "dataset", "split", "source_file", "text_raw", "text",
341     "text_lower",
342     "bin_label", "main_label", "n_active", "n_words", "n_chars", "avg_word_len",
343     "upper_ratio", "exclaim_cnt", "question_cnt", "length_bin"
344 ]].copy().rename(columns={"bin_label": "label", "main_label": "source_emotion"})
345
346 go_multi.to_csv(data_dir / "clean_goemotions_multiclass.csv", index=False,
347                 encoding="utf-8-sig")
348 go_bin.to_csv(data_dir / "clean_goemotions_binary.csv", index=False,
349               encoding="utf-8-sig")
350
351 go_multi.head(), go_bin.head()
352
353 req_cols_bin = [
354     "id", "row_id", "dataset", "split", "source_file", "text_raw", "text",
355     "text_lower",
356     "label", "n_words", "n_chars", "avg_word_len", "upper_ratio",
357     "exclaim_cnt", "question_cnt", "length_bin"
358 ]
359
360 valid_rows = []
361 valid_rows.append({
362     "check": "bin_columns_match",
363     "result": list(imdb[req_cols_bin].columns) == list(go_bin[req_cols_bin].columns),
364     "detail": "IMDb and GoEmotions binary tables share the same required columns."
365 })

```

```

360 })
361 valid_rows.append({
362     "check": "bin_label_values_imdb",
363     "result": set(imdb.loc[imdb["split"] != "unlabeled",
364         "label"].dropna().unique().tolist()) == {0.0, 1.0},
365     "detail": "IMDb labeled rows use 0 and 1 only."
366 })
367 valid_rows.append({
368     "check": "bin_label_values_go",
369     "result": set(go_bin["label"].dropna().unique().tolist()) == {0.0, 1.0},
370     "detail": "GoEmotions binary rows use 0 and 1 only."
371 })
372 valid_rows.append({
373     "check": "no_null_text_imdb",
374     "result": int(imdb["text"].isna().sum()) == 0,
375     "detail": "IMDb clean table has no null text."
376 })
377 valid_rows.append({
378     "check": "no_null_text_go_bin",
379     "result": int(go_bin["text"].isna().sum()) == 0,
380     "detail": "GoEmotions binary clean table has no null text."
381 })
382 valid_rows.append({
383     "check": "text_lower_exists",
384     "result": all("text_lower" in df.columns for df in [imdb, go_multi, go_bin]),
385     "detail": "All clean tables keep a lowercased text column."
386 })
387 valid_rows.append({
388     "check": "length_bin_exists",
389     "result": all("length_bin" in df.columns for df in [imdb, go_multi, go_bin]),
390     "detail": "All clean tables keep a persistent length bin column."
391 })
392 valid_rows.append({
393     "check": "config_saved",
394     "result": (prep_dir / "config.json").exists(),
395     "detail": "External config file exists."
396 })
397 valid_rows.append({
398     "check": "zero_label_logged",
399     "result": (prep_dir / "go_zero_label.csv").exists(),
400     "detail": "Rows with zero active GoEmotions labels are logged."
401 })
402 train_text = set(imdb.loc[imdb["split"] == "train", "text"])
403 test_text = set(imdb.loc[imdb["split"] == "test", "text"])
404 valid_rows.append({
405     "check": "imdb_train_test_overlap_text",
406     "result": len(train_text.intersection(test_text)) == 0,
407     "detail": f"Shared texts across IMDb train and test:
408         {len(train_text.intersection(test_text))}"
409 })
410 go_train = set(go_bin.loc[go_bin["split"].astype(str).str.contains("train",
411     case=False, na=False), "text"])
412 go_test = set(go_bin.loc[go_bin["split"].astype(str).str.contains("test", case=False,
413     na=False), "text"])
414 valid_rows.append({
415     "check": "go_train_test_overlap_text",
416     "result": len(go_train.intersection(go_test)) == 0,
417     "detail": f"Shared texts across GoEmotions train and test-like splits:
418         {len(go_train.intersection(go_test))}"
419 })
420 valid = pd.DataFrame(valid_rows)
421 valid.to_csv(prep_dir / "dataset_validation_report.csv", index=False,
422     encoding="utf-8-sig")
423 valid
424
425 profile = pd.concat([
426     profile_table(imdb, "imdb"),
427     profile_table(go_multi, "go_multi"),

```

```

422     profile_table(go_bin, "go_bin")
423 ], ignore_index=True)
424 profile.to_csv(prepare_dir / "corpus_profile_report.csv", index=False,
425               encoding="utf-8-sig")
426
427 ready = pd.DataFrame([
428     {"item": "aligned_binary_columns", "status": bool(valid.loc[valid['check'] ==
429     'bin_columns_match', 'result'].iloc[0])},
430     {"item": "aligned_binary_labels", "status": bool(valid.loc[valid['check'] ==
431     'bin_label_values_imdb', 'result'].iloc[0]) and bool(valid.loc[valid['check']
432     == 'bin_label_values_go', 'result'].iloc[0])},
433     {"item": "text_is_string_cleaned", "status": int(imdb['text'].isna().sum()) == 0
434     and int(go_bin['text'].isna().sum()) == 0},
435     {"item": "text_lower_saved", "status": all('text_lower' in df.columns for df in
436     [imdb, go_multi, go_bin])},
437     {"item": "config_externalized", "status": (prepare_dir / 'config.json').exists()},
438     {"item": "meta_columns_saved", "status": all(set(['dataset', 'split',
439     'source_file']).issubset(df.columns) for df in [imdb, go_multi, go_bin])},
440     {"item": "length_bin_saved", "status": all('length_bin' in df.columns for df in
441     [imdb, go_multi, go_bin])},
442     {"item": "go_zero_label_checked", "status": (prepare_dir /
443     'go_zero_label.csv').exists()},
444     {"item": "imdb_unlabeled_kept", "status": 'unlabeled' in
445     set(imdb['split'].astype(str))},
446     {"item": "imdb_train_test_no_overlap", "status": bool(valid.loc[valid['check'] ==
447     'imdb_train_test_overlap_text', 'result'].iloc[0])},
448     {"item": "go_train_test_no_overlap", "status": bool(valid.loc[valid['check'] ==
449     'go_train_test_overlap_text', 'result'].iloc[0])}
450 ])
451 ready.to_csv(prepare_dir / "readiness_check.csv", index=False, encoding="utf-8-sig")
452
453 profile.head(), ready
454
455 lines = [
456     "Data preparation finished.",
457     f"IMDb clean file: {data_dir / 'clean_imdb_binary.csv'}",
458     f"GoEmotions multiclass file: {data_dir / 'clean_goemotions_multiclass.csv'}",
459     f"GoEmotions binary file: {data_dir / 'clean_goemotions_binary.csv'}",
460     f"Corpus profile report: {prepare_dir / 'corpus_profile_report.csv'}",
461     f"Validation report: {prepare_dir / 'dataset_validation_report.csv'}",
462     f"Readiness report: {prepare_dir / 'readiness_check.csv'}",
463     f"Zero-label log: {prepare_dir / 'go_zero_label.csv'}",
464     f"Config file: {prepare_dir / 'config.json'}"
465 ]
466
467 with open(prepare_dir / "process_log.txt", "w", encoding="utf-8") as f:
468     f.write("\n".join(lines))
469
470 "\n".join(lines)
471
472 for fp in [
473     data_dir / "clean_imdb_binary.csv",
474     data_dir / "clean_goemotions_multiclass.csv",
475     data_dir / "clean_goemotions_binary.csv",
476     prepare_dir / "config.json",
477     prepare_dir / "corpus_profile_report.csv",
478     prepare_dir / "dataset_validation_report.csv",
479     prepare_dir / "readiness_check.csv",
480     prepare_dir / "go_zero_label.csv",
481     prepare_dir / "process_log.txt",
482 ]:
483     print(fp, "->", fp.exists())

```

Listing 1. Project code

step2_features_final.ipynb

```
1 %pip install -q numpy pandas scipy scikit-learn statsmodels
2
3 from pathlib import Path
4 import json
5 import re
6 import string
7
8 import numpy as np
9 import pandas as pd
10 import statsmodels.api as sm
11
12 from scipy.spatial.distance import jensenshannon
13 from scipy.stats import ks_2samp
14 from sklearn.ensemble import RandomForestClassifier
15 from sklearn.linear_model import LogisticRegression
16 from sklearn.metrics import f1_score
17 from sklearn.pipeline import Pipeline
18 from sklearn.preprocessing import StandardScaler
19 from statsmodels.stats.multitest import multipletests
20 from statsmodels.stats.outliers_influence import variance_inflation_factor
21
22 root = Path.cwd()
23 data_dir = root / 'data'
24 prep_dir = root / 'outputs' / 'data_prep'
25 step2_dir = root / 'outputs' / 'step2_features'
26 step2_dir.mkdir(parents=True, exist_ok=True)
27
28 config_path = prep_dir / 'config.json'
29 if not config_path.exists():
30     raise FileNotFoundError(f'Missing config file: {config_path}')
31
32 with open(config_path, 'r', encoding='utf-8') as f:
33     step1_cfg = json.load(f)
34
35 imdb_path = data_dir / 'clean_imdb_binary.csv'
36 go_bin_path = data_dir / 'clean_goemotions_binary.csv'
37 go_multi_path = data_dir / 'clean_goemotions_multiclass.csv'
38
39 for fp in [imdb_path, go_bin_path, go_multi_path]:
40     if not fp.exists():
41         raise FileNotFoundError(f'Missing Step 1 clean file: {fp}')
42
43 cfg2 = {
44     'seed': 42,
45     'dominant_value_ratio_max': 0.99,
46     'variance_eps': 1e-12,
47     'jsd_continuous_bins': 30,
48     'fdr_alpha': 0.05,
49
50     # Quantile-based stability thresholds
51     'use_quantile_thresholds': True,
52     'ks_keep_quantile': 0.25,          # keep lowest 25% KS
53     'jsd_keep_quantile': 0.50,        # keep lower 50% JSD
54     'cliff_keep_quantile': 0.25,      # keep lowest 25% |Cliff's delta|
55
56     # fallback fixed thresholds, kept for comparison / backup
57     'ks_max': 0.30,
58     'jsd_max': 0.10,
59     'cliffs_delta_max': 0.25,
60
61     'l1_coef_min': 1e-4,
62     'rf_importance_min': 1e-4,
63     'vif_max': 10.0,
64     'rf_estimators': 300,
65     'rf_max_depth': None,
```

```

66 }
67 print('root:', root)
68 print('step2_dir:', step2_dir)
69 print(json.dumps(cfg2, indent=2))
70
71 imdb = pd.read_csv(imdb_path)
72 go_bin = pd.read_csv(go_bin_path)
73 go_multi = pd.read_csv(go_multi_path)
74
75 required_cols = [
76     'id', 'dataset', 'split', 'text', 'text_lower', 'label',
77     'n_words', 'n_chars', 'avg_word_len', 'upper_ratio',
78     'exclaim_cnt', 'question_cnt', 'length_bin'
79 ]
80
81 def check_required_columns(df, name):
82     missing = [c for c in required_cols if c not in df.columns]
83     if missing:
84         raise ValueError(f'{name} missing required columns: {missing}')
85
86 check_required_columns(imdb, 'imdb')
87 check_required_columns(go_bin, 'go_bin')
88 check_required_columns(go_multi, 'go_multi')
89
90 imdb['domain'] = 'imdb'
91 go_bin['domain'] = 'go'
92 go_multi['domain'] = 'go'
93 imdb['task'] = 'binary'
94 go_bin['task'] = 'binary'
95 go_multi['task'] = 'multiclass'
96
97 imdb.shape, go_bin.shape, go_multi.shape
98
99 paths2 = {
100     'feature_registry': step2_dir / 'feature_registry.csv',
101     'feature_registry_snapshot': step2_dir / 'feature_registry_snapshot.csv',
102     'all_features_raw': step2_dir / 'all_features_raw.csv',
103     'all_features_stats': step2_dir / 'all_features_stats.csv',
104     'stable_features_final': step2_dir / 'stable_features_final.csv',
105     'feature_diagnostics': step2_dir / 'feature_diagnostics.csv',
106     'step2_log': step2_dir / 'step2_log.txt',
107     'step2_config': step2_dir / 'step2_config.json',
108 }
109
110 with open(paths2['step2_config'], 'w', encoding='utf-8') as f:
111     json.dump({'cfg2': cfg2, 'paths2': {k: str(v) for k, v in paths2.items()}}, f,
112               ensure_ascii=False, indent=2)
113
114 {k: str(v) for k, v in paths2.items()}
115
116 registry_rows = []
117
118 def register_feature(feature_name, feature_group, feature_type, source_text,
119                     is_count=False, is_normalized=False, use_for_stability=True,
120                     notes=''):
121     registry_rows.append({
122         'feature_name': feature_name,
123         'feature_group': feature_group,
124         'feature_type': feature_type,
125         'source_text': source_text,
126         'is_count': bool(is_count),
127         'is_normalized': bool(is_normalized),
128         'use_for_stability': bool(use_for_stability),
129         'low_variance_filtered': False,
130         'selected_final': False,
131         'drop_reason': '',
132         'notes': notes,
133     })

```

```

133
134 def registry_df():
135     return pd.DataFrame(registry_rows).drop_duplicates(subset=['feature_name'],
136                                                         keep='last').reset_index(drop=True)
137
138 def update_registry(feature_name, **kwargs):
139     for row in registry_rows:
140         if row['feature_name'] == feature_name:
141             row.update(kwargs)
142
143 TOKEN_RE = re.compile(r"\b\w+\b")
144 SENT_RE = re.compile(r"[.!?]+")
145
146 NEGATIONS = {'not', 'no', 'never', 'none', 'nobody', 'nothing', 'neither', 'nor',
147              'n't'}
148 INTENSIFIERS = {'very', 'really', 'extremely', 'totally', 'absolutely', 'highly',
149                'so', 'super'}
150 DOWNTONERS = {'slightly', 'somewhat', 'barely', 'hardly', 'partly', 'fairly', 'rather'}
151 POSITIVE_LEXICON = {'good', 'great', 'excellent', 'amazing', 'wonderful', 'love',
152                     'liked', 'happy', 'joy', 'fun'}
153 NEGATIVE_LEXICON = {'bad', 'terrible', 'awful', 'hate', 'hated', 'sad', 'angry',
154                     'fear', 'boring', 'worse'}
155 NEUTRAL_OTHER_LEXICON = {'maybe', 'perhaps', 'surprised', 'curious', 'realize',
156                           'realized', 'guess', 'wonder'}
157
158 def tokenize(text):
159     if not isinstance(text, str):
160         return []
161     return TOKEN_RE.findall(text)
162
163 def split_sentences(text):
164     if not isinstance(text, str):
165         return []
166     parts = [p.strip() for p in SENT_RE.split(text) if p.strip()]
167     return parts
168
169 def safe_div(a, b):
170     return float(a) / float(b) if b else 0.0
171
172 def per_100_words(count, n_words):
173     return safe_div(count * 100.0, n_words)
174
175 def type_token_ratio(tokens):
176     return safe_div(len(set(tokens)), len(tokens))
177
178 def long_word_ratio(tokens, min_len=7):
179     return safe_div(sum(len(t) >= min_len for t in tokens), len(tokens))
180
181 def short_word_ratio(tokens, max_len=3):
182     return safe_div(sum(len(t) <= max_len for t in tokens), len(tokens))
183
184 def all_caps_token_ratio(text):
185     toks = tokenize(text)
186     if not toks:
187         return 0.0
188     return safe_div(sum(t.isupper() and len(t) > 1 for t in toks), len(toks))
189
190 def repeat_char_ratio(text):
191     if not isinstance(text, str) or not text:
192         return 0.0
193     repeat_hits = len(re.findall(r'(\.)\1{2,}', text))
194     return safe_div(repeat_hits, len(text))
195
196 def punct_density(text):
197     if not isinstance(text, str) or not text:
198         return 0.0
199     punct = sum(ch in string.punctuation for ch in text)
200     return safe_div(punct, len(text))

```

```

195
196 def count_lexicon(tokens, lexicon):
197     return sum(t in lexicon for t in tokens)
198
199 def emotion_entropy(counts):
200     arr = np.array(counts, dtype=float)
201     s = arr.sum()
202     if s <= 0:
203         return 0.0
204     p = arr / s
205     p = p[p > 0]
206     return float(-(p * np.log2(p)).sum())
207
208 def cliffs_delta(x, y):
209     x = np.asarray(pd.Series(x).dropna(), dtype=float)
210     y = np.asarray(pd.Series(y).dropna(), dtype=float)
211     if len(x) == 0 or len(y) == 0:
212         return np.nan
213     xy = np.subtract.outer(x, y)
214     return float((np.sum(xy > 0) - np.sum(xy < 0)) / (len(x) * len(y)))
215
216 def probability_vector_discrete(x, y):
217     sx = pd.Series(x).dropna()
218     sy = pd.Series(y).dropna()
219     levels = sorted(set(sx.tolist()) | set(sy.tolist()))
220     if not levels:
221         return np.array([1.0]), np.array([1.0])
222     px = sx.value_counts().reindex(levels, fill_value=0).to_numpy(dtype=float)
223     py = sy.value_counts().reindex(levels, fill_value=0).to_numpy(dtype=float)
224     px = px / px.sum() if px.sum() else np.ones_like(px) / len(px)
225     py = py / py.sum() if py.sum() else np.ones_like(py) / len(py)
226     return px, py
227
228 def probability_vector_continuous(x, y, n_bins=30):
229     sx = pd.Series(x).dropna().astype(float)
230     sy = pd.Series(y).dropna().astype(float)
231     if len(sx) == 0 or len(sy) == 0:
232         return np.array([1.0]), np.array([1.0])
233     global_min = float(min(sx.min(), sy.min()))
234     global_max = float(max(sx.max(), sy.max()))
235     if global_min == global_max:
236         return np.array([1.0]), np.array([1.0])
237     bins = np.linspace(global_min, global_max, n_bins + 1)
238     px, _ = np.histogram(sx, bins=bins)
239     py, _ = np.histogram(sy, bins=bins)
240     px = px.astype(float)
241     py = py.astype(float)
242     px = px / px.sum() if px.sum() else np.ones_like(px) / len(px)
243     py = py / py.sum() if py.sum() else np.ones_like(py) / len(py)
244     return px, py
245
246 def compute_jsd(x, y, feature_type):
247     if feature_type == 'continuous':
248         px, py = probability_vector_continuous(x, y,
249                                             n_bins=config2['jsd_continuous_bins'])
250     else:
251         px, py = probability_vector_discrete(x, y)
252     return float(jensenshannon(px, py, base=2.0) ** 2)
253
254 base_feature_specs = [
255     ('n_words', 'structure', 'continuous', 'text', True, False, True, 'Step 1 base
256     field'),
257     ('n_chars', 'structure', 'continuous', 'text', True, False, True, 'Step 1 base
258     field'),
259     ('avg_word_len', 'structure', 'continuous', 'text', False, False, True, 'Step 1
260     base field'),
261     ('upper_ratio', 'style', 'continuous', 'text', False, False, True, 'Step 1 base
262     field'),

```

```

258     ('exclaim_cnt', 'style', 'discrete', 'text', True, False, True, 'Step 1 base
        field'),
259     ('question_cnt', 'style', 'discrete', 'text', True, False, True, 'Step 1 base
        field'),
260     ('sentence_count', 'structure', 'discrete', 'text', True, False, True, ''),
261     ('avg_sentence_len_words', 'structure', 'continuous', 'text', False, False, True,
        ''),
262     ('type_token_ratio', 'structure', 'continuous', 'text_lower', False, False, True,
        ''),
263     ('long_word_ratio', 'structure', 'continuous', 'text_lower', False, False, True,
        ''),
264     ('short_word_ratio', 'structure', 'continuous', 'text_lower', False, False, True,
        ''),
265     ('all_caps_token_ratio', 'style', 'continuous', 'text', False, False, True, ''),
266     ('repeat_char_ratio', 'style', 'continuous', 'text', False, False, True, ''),
267     ('punct_density', 'style', 'continuous', 'text', False, False, True, ''),
268     ('exclaim_per_100w', 'style', 'continuous', 'text', True, True, True, ''),
269     ('question_per_100w', 'style', 'continuous', 'text', True, True, True, ''),
270     ('negation_cnt', 'lexical', 'discrete', 'text_lower', True, False, True, ''),
271     ('negation_per_100w', 'lexical', 'continuous', 'text_lower', True, True, True, ''),
272     ('intensifier_cnt', 'lexical', 'discrete', 'text_lower', True, False, True, ''),
273     ('intensifier_per_100w', 'lexical', 'continuous', 'text_lower', True, True, True,
        ''),
274     ('downtoner_cnt', 'lexical', 'discrete', 'text_lower', True, False, True, ''),
275     ('downtoner_per_100w', 'lexical', 'continuous', 'text_lower', True, True, True,
        ''),
276     ('positive_lexicon_ratio', 'emotion', 'continuous', 'text_lower', False, True,
        True, ''),
277     ('negative_lexicon_ratio', 'emotion', 'continuous', 'text_lower', False, True,
        True, ''),
278     ('neutral_other_ratio', 'emotion', 'continuous', 'text_lower', False, True, True,
        ''),
279     ('emotion_entropy', 'emotion', 'continuous', 'text_lower', False, False, True, ''),
280 ]
281
282 for spec in base_feature_specs:
283     register_feature(*spec)
284
285 registry_df().head()
286
287 def extract_features(df):
288     out = df.copy()
289
290     tokens_lower = out['text_lower'].fillna('').apply(tokenize)
291     sentences = out['text'].fillna('').apply(split_sentences)
292
293     out['sentence_count'] = sentences.apply(len)
294     out['avg_sentence_len_words'] = [safe_div(len(toks), len(sents)) if len(sents)
        else 0.0 for toks, sents in zip(tokens_lower, sentences)]
295     out['type_token_ratio'] = tokens_lower.apply(type_token_ratio)
296     out['long_word_ratio'] = tokens_lower.apply(long_word_ratio)
297     out['short_word_ratio'] = tokens_lower.apply(short_word_ratio)
298
299     out['all_caps_token_ratio'] = out['text'].apply(all_caps_token_ratio)
300     out['repeat_char_ratio'] = out['text'].apply(repeat_char_ratio)
301     out['punct_density'] = out['text'].apply(punct_density)
302     out['exclaim_per_100w'] = [per_100_words(c, n) for c, n in zip(out['exclaim_cnt'],
        out['n_words'])]
303     out['question_per_100w'] = [per_100_words(c, n) for c, n in
        zip(out['question_cnt'], out['n_words'])]
304
305     out['negation_cnt'] = tokens_lower.apply(lambda toks: count_lexicon(toks,
        NEGATIONS))
306     out['negation_per_100w'] = [per_100_words(c, n) for c, n in
        zip(out['negation_cnt'], out['n_words'])]
307     out['intensifier_cnt'] = tokens_lower.apply(lambda toks: count_lexicon(toks,
        INTENSIFIERS))

```

```

308     out['intensifier_per_100w'] = [per_100_words(c, n) for c, n in
        zip(out['intensifier_cnt'], out['n_words'])]
309     out['downtoner_cnt'] = tokens_lower.apply(lambda toks: count_lexicon(toks,
        DOWNTONERS))
310     out['downtoner_per_100w'] = [per_100_words(c, n) for c, n in
        zip(out['downtoner_cnt'], out['n_words'])]
311
312     pos_cnt = tokens_lower.apply(lambda toks: count_lexicon(toks, POSITIVE_LEXICON))
313     neg_cnt = tokens_lower.apply(lambda toks: count_lexicon(toks, NEGATIVE_LEXICON))
314     neu_cnt = tokens_lower.apply(lambda toks: count_lexicon(toks,
        NEUTRAL_OTHER_LEXICON))
315
316     out['positive_lexicon_ratio'] = [safe_div(c, n) for c, n in zip(pos_cnt,
        out['n_words'])]
317     out['negative_lexicon_ratio'] = [safe_div(c, n) for c, n in zip(neg_cnt,
        out['n_words'])]
318     out['neutral_other_ratio'] = [safe_div(c, n) for c, n in zip(neu_cnt,
        out['n_words'])]
319     out['emotion_entropy'] = [emotion_entropy([p, n, u]) for p, n, u in zip(pos_cnt,
        neg_cnt, neu_cnt)]
320
321     return out
322
323     imdb_feat = extract_features(imdb)
324     go_bin_feat = extract_features(go_bin)
325     go_multi_feat = extract_features(go_multi)
326
327     all_features = pd.concat([imdb_feat, go_bin_feat, go_multi_feat], ignore_index=True)
328     all_features.to_csv(paths2['all_features_raw'], index=False, encoding='utf-8-sig')
329     registry_df().to_csv(paths2['feature_registry_snapshot'], index=False,
        encoding='utf-8-sig')
330
331     all_features.shape
332
333     feature_list = registry_df()['feature_name'].tolist()
334     lv_rows = []
335
336     binary_pool = pd.concat([
337         imdb_feat[imdb_feat['split'] != 'unlabeled'].copy(),
338         go_bin_feat.copy()
339     ], ignore_index=True)
340
341     for feat in feature_list:
342         series = pd.Series(binary_pool[feat]).dropna()
343         variance = float(series.var()) if len(series) else np.nan
344         top_ratio = float(series.value_counts(normalize=True, dropna=False).iloc[0]) if
            len(series) else np.nan
345         low_var = (not np.isnan(variance) and variance <= cfg2['variance_eps']) or (not
            np.isnan(top_ratio) and top_ratio >= cfg2['dominant_value_ratio_max'])
346         lv_rows.append({'feature_name': feat, 'variance': variance, 'top_value_ratio':
            top_ratio, 'low_variance_filtered': low_var})
347         if low_var:
348             update_registry(feat, low_variance_filtered=True, use_for_stability=False,
                drop_reason='low_variance_or_dominant_value')
349
350     lv_df = pd.DataFrame(lv_rows)
351     lv_df.head()
352
353     reg_now = registry_df()
354     stable_pool = reg_now.loc[reg_now['use_for_stability'], ['feature_name',
        'feature_group', 'feature_type']].copy()
355
356     imdb_bin = imdb_feat[imdb_feat['split'] != 'unlabeled'].copy()
357     go_bin_only = go_bin_feat.copy()
358
359     stat_rows = []
360
361     for _, row in stable_pool.iterrows():

```

```

362     feat = row['feature_name']
363     fgroup = row['feature_group']
364     ftype = row['feature_type']
365     x_go = pd.Series(go_bin_only[feat]).dropna()
366     x_imdb = pd.Series(imdb_bin[feat]).dropna()
367
368     ks_res = ks_2samp(x_go, x_imdb)
369
370     stat_rows.append({
371         'feature_name': feat,
372         'feature_group': fgroup,
373         'feature_type': ftype,
374         'n_go': int(len(x_go)),
375         'n_imdb': int(len(x_imdb)),
376         'ks_stat': float(ks_res.statistic),
377         'p_value_raw': float(ks_res.pvalue),
378         'cliffs_delta': float(cliffs_delta(x_go, x_imdb)),
379         'jsd': float(compute_jsd(x_go, x_imdb, ftype)),
380         'is_stable_candidate': False,
381         'selected_final': False,
382         'drop_reason': '',
383     })
384
385     stats_df = pd.DataFrame(stat_rows)
386
387     if len(stats_df):
388         reject, p_fdr, _, _ = multipletests(
389             stats_df['p_value_raw'].to_numpy(),
390             alpha=cfg2['fdr_alpha'],
391             method='fdr_bh'
392         )
393         stats_df['p_value_fdr'] = p_fdr
394         stats_df['reject_fdr'] = reject
395     else:
396         stats_df['p_value_fdr'] = np.nan
397         stats_df['reject_fdr'] = False
398
399     # Keep FDR only as a diagnostic signal, not a hard exclusion rule.
400     stats_df['fdr_warning'] = stats_df['reject_fdr']
401
402     # Absolute Cliff's delta for ranking stability
403     stats_df['abs_cliffs_delta'] = stats_df['cliffs_delta'].abs()
404
405     if cfg2.get('use_quantile_thresholds', False):
406         ks_cutoff = stats_df['ks_stat'].quantile(cfg2['ks_keep_quantile'])
407         jsd_cutoff = stats_df['jsd'].quantile(cfg2['jsd_keep_quantile'])
408         cliff_cutoff = stats_df['abs_cliffs_delta'].quantile(cfg2['cliff_keep_quantile'])
409
410         stats_df['is_stable_candidate'] = (
411             (stats_df['ks_stat'] <= ks_cutoff) &
412             (stats_df['jsd'] <= jsd_cutoff) &
413             (stats_df['abs_cliffs_delta'] <= cliff_cutoff)
414         )
415
416         stats_df['threshold_mode'] = 'quantile'
417         stats_df['ks_cutoff_used'] = float(ks_cutoff)
418         stats_df['jsd_cutoff_used'] = float(jsd_cutoff)
419         stats_df['cliff_cutoff_used'] = float(cliff_cutoff)
420
421     else:
422         stats_df['is_stable_candidate'] = (
423             (stats_df['ks_stat'] <= cfg2['ks_max']) &
424             (stats_df['jsd'] <= cfg2['jsd_max']) &
425             (stats_df['abs_cliffs_delta'] <= cfg2['cliffs_delta_max'])
426         )
427
428         stats_df['threshold_mode'] = 'fixed'
429         stats_df['ks_cutoff_used'] = float(cfg2['ks_max'])

```

```

430     stats_df['jsd_cutoff_used'] = float(cfg2['jsd_max'])
431     stats_df['cliff_cutoff_used'] = float(cfg2['cliffs_delta_max'])
432
433     unstable_mask = ~stats_df['is_stable_candidate']
434     stats_df.loc[unstable_mask, 'drop_reason'] = 'distribution_shift_too_large'
435
436     for _, row in stats_df.iterrows():
437         if row['is_stable_candidate']:
438             update_registry(row['feature_name'], drop_reason='')
439         else:
440             update_registry(row['feature_name'],
441                             drop_reason='distribution_shift_too_large')
442
443     print('threshold_mode:', stats_df['threshold_mode'].iloc[0] if len(stats_df) else 'NA')
444     print('ks_cutoff_used:', stats_df['ks_cutoff_used'].iloc[0] if len(stats_df) else 'NA')
445     print('jsd_cutoff_used:', stats_df['jsd_cutoff_used'].iloc[0] if len(stats_df) else
446           'NA')
447     print('cliff_cutoff_used:', stats_df['cliff_cutoff_used'].iloc[0] if len(stats_df)
448           else 'NA')
449     print('stable candidates:', int(stats_df['is_stable_candidate'].sum()))
450
451     stats_df.sort_values(['is_stable_candidate', 'jsd', 'ks_stat'], ascending=[False,
452                                     True, True]).head(20)
453
454     candidate_features = stats_df.loc[stats_df['is_stable_candidate'],
455                                     'feature_name'].tolist()
456
457     go_train = go_bin_feat[go_bin_feat['split'].astype(str).str.lower() == 'train'].copy()
458     go_valid = go_bin_feat[go_bin_feat['split'].astype(str).str.lower() ==
459                           'validation'].copy()
460
461     if not len(go_train):
462         raise ValueError('No GoEmotions train rows found for predictive validation.')
463
464     y_train = go_train['label'].astype(int)
465     y_valid = go_valid['label'].astype(int) if len(go_valid) else pd.Series(dtype=int)
466
467     pred_rows = []
468
469     if candidate_features:
470         X_train = go_train[candidate_features].copy()
471
472         l1_pipe = Pipeline([
473             ('scaler', StandardScaler()),
474             ('clf', LogisticRegression(
475                 penalty='l1',
476                 solver='liblinear',
477                 random_state=cfg2['seed'],
478                 max_iter=2000,
479             ))
480         ])
481         l1_pipe.fit(X_train, y_train)
482         l1_coef = np.abs(l1_pipe.named_steps['clf'].coef_.ravel())
483
484         rf = RandomForestClassifier(
485             n_estimators=cfg2['rf_estimators'],
486             max_depth=cfg2['rf_max_depth'],
487             random_state=cfg2['seed'],
488             n_jobs=-1,
489         )
490         rf.fit(X_train, y_train)
491         rf_importance = rf.feature_importances_
492
493     for feat, coef_val, imp_val in zip(candidate_features, l1_coef, rf_importance):
494         has_signal = (coef_val > cfg2['l1_coef_min']) or (imp_val >
495                 cfg2['rf_importance_min'])
496         pred_rows.append({
497             'feature_name': feat,

```

```

491         'abs_l1_coef': float(coef_val),
492         'rf_importance': float(imp_val),
493         'has_predictive_signal': bool(has_signal),
494     })
495
496 pred_df = pd.DataFrame(
497     pred_rows,
498     columns=['feature_name', 'abs_l1_coef', 'rf_importance', 'has_predictive_signal']
499 )
500
501 print('candidate_features:', len(candidate_features))
502 print('pred_df shape:', pred_df.shape)
503 print('pred_df columns:', pred_df.columns.tolist())
504
505 pred_df.head()
506
507 baseline_features = ['n_words', 'n_chars', 'avg_word_len', 'upper_ratio',
508                     'exclaim_cnt', 'question_cnt']
509 baseline_features = [f for f in baseline_features if f in go_train.columns]
510 stable_signal_features = pred_df.loc[pred_df['has_predictive_signal'],
511                                   'feature_name'].tolist() if len(pred_df) else []
512 combined_features = sorted(set(baseline_features) | set(stable_signal_features))
513
514 compare_rows = []
515
516 def run_validation_model(features):
517     if not features or len(go_valid) == 0:
518         return np.nan
519     pipe = Pipeline([
520         ('scaler', StandardScaler()),
521         ('clf', LogisticRegression(
522             penalty='l2',
523             solver='liblinear',
524             random_state=cfg2['seed'],
525             max_iter=2000,
526         ))
527     ])
528     pipe.fit(go_train[features], y_train)
529     pred = pipe.predict(go_valid[features])
530     return float(f1_score(y_valid, pred))
531
532 compare_rows.append({'model': 'baseline', 'features': len(baseline_features),
533                    'validation_f1': run_validation_model(baseline_features)})
534 compare_rows.append({'model': 'stable_only', 'features': len(stable_signal_features),
535                    'validation_f1': run_validation_model(stable_signal_features)})
536 compare_rows.append({'model': 'stable_plus_base', 'features': len(combined_features),
537                    'validation_f1': run_validation_model(combined_features)})
538
539 compare_df = pd.DataFrame(compare_rows)
540
541 if 'feature_name' not in pred_df.columns:
542     pred_df = pd.DataFrame(
543         columns=['feature_name', 'abs_l1_coef', 'rf_importance',
544                'has_predictive_signal']
545     )
546
547 stats_df = stats_df.merge(pred_df, on='feature_name', how='left')
548 stats_df['has_predictive_signal'] = stats_df['has_predictive_signal'].fillna(False)
549 stats_df['selected_final'] = stats_df['is_stable_candidate'] &
550     stats_df['has_predictive_signal']
551
552 stats_df.loc[
553     stats_df['is_stable_candidate'] & (~stats_df['has_predictive_signal']),
554     'drop_reason'
555 ] = 'no_predictive_gain'
556
557 for _, row in stats_df.iterrows():

```

```

552     update_registry(
553         row['feature_name'],
554         selected_final=bool(row['selected_final']),
555         drop_reason=row['drop_reason'] if isinstance(row['drop_reason'], str) else ''
556     )
557
558 stable_final = stats_df.loc[stats_df['selected_final']].copy()
559
560 print('stable_final shape:', stable_final.shape)
561 stable_final.head()
562
563 diag_rows = []
564 final_feature_list = stable_final['feature_name'].tolist()
565
566 if len(final_feature_list) >= 2:
567     X_corr = go_train[final_feature_list].copy()
568     corr_matrix = X_corr.corr(numeric_only=True)
569     cols = corr_matrix.columns.tolist()
570     for i in range(len(cols)):
571         for j in range(i + 1, len(cols)):
572             if abs(corr_matrix.iloc[i, j]) >= 0.90:
573                 diag_rows.append({'check': 'high_correlation_pair', 'feature_name':
574                                     cols[i], 'value': float(corr_matrix.iloc[i, j]), 'detail':
575                                     f'{cols[i]} vs {cols[j]}'})
576
577 removed_vif = []
578 current_features = final_feature_list.copy()
579
580 while len(current_features) >= 2:
581     X_const = sm.add_constant(go_train[current_features].astype(float),
582                               has_constant='add')
583     vif_vals = []
584     for i, col in enumerate(X_const.columns):
585         if col == 'const':
586             continue
587         vif_vals.append((col, float(variance_inflation_factor(X_const.values, i))))
588     vif_df = pd.DataFrame(vif_vals, columns=['feature_name',
589                                             'vif']).sort_values('vif', ascending=False)
590     top_feat = str(vif_df.iloc[0]['feature_name'])
591     top_vif = float(vif_df.iloc[0]['vif'])
592     if top_vif <= cfg2['vif_max']:
593         break
594     removed_vif.append({'feature_name': top_feat, 'vif': top_vif})
595     update_registry(top_feat, selected_final=False, drop_reason='collinearity')
596     current_features.remove(top_feat)
597
598 final_after_vif = current_features.copy()
599
600 if len(final_after_vif) >= 1:
601     X_const = sm.add_constant(go_train[final_after_vif].astype(float),
602                               has_constant='add')
603     vif_final_df = []
604     for i, col in enumerate(X_const.columns):
605         if col == 'const':
606             continue
607         vif_final_df.append({'feature_name': col, 'vif':
608                               float(variance_inflation_factor(X_const.values, i))})
609     vif_final_df = pd.DataFrame(vif_final_df)
610 else:
611     vif_final_df = pd.DataFrame(columns=['feature_name', 'vif'])
612
613 for row in removed_vif:
614     diag_rows.append({'check': 'iterative_vif_removed', 'feature_name':
615                       row['feature_name'], 'value': row['vif'], 'detail': 'removed in iterative VIF
616                               elimination'})
617
618 for _, row in vif_final_df.iterrows():
619     diag_rows.append({'check': 'final_vif', 'feature_name': row['feature_name'],
620                       'value': row['vif'], 'detail': 'remaining feature after iterative

```

```

        elimination'}}
611
612 diagnostics_df = pd.DataFrame(diag_rows)
613 final_registry = registry_df()
614 final_stable_features = final_registry.loc[final_registry['selected_final'],
        ['feature_name']].merge(stats_df, on='feature_name', how='left')
615 final_stable_features =
        final_stable_features[final_stable_features['feature_name'].isin(final_after_vif)].copy()
616 final_stable_features['selected_final'] = True
617
618 final_stable_features.head()
619
620 final_registry = registry_df()
621 stats_out = stats_df.copy()
622 stats_out.loc[~stats_out['feature_name'].isin(final_after_vif), 'selected_final'] =
        False
623 stats_out.loc[(stats_out['selected_final']) &
        (stats_out['feature_name'].isin(final_after_vif)), 'drop_reason'] = ''
624 stats_out.loc[(~stats_out['feature_name'].isin(final_after_vif)) &
        (stats_out['drop_reason'] == ''), 'drop_reason'] = 'collinearity'
625
626 final_registry.to_csv(paths2['feature_registry'], index=False, encoding='utf-8-sig')
627 stats_out.to_csv(paths2['all_features_stats'], index=False, encoding='utf-8-sig')
628 final_stable_features.to_csv(paths2['stable_features_final'], index=False,
        encoding='utf-8-sig')
629 diagnostics_df.to_csv(paths2['feature_diagnostics'], index=False, encoding='utf-8-sig')
630
631 checks = {
632     'feature_registry_exists': paths2['feature_registry'].exists(),
633     'feature_registry_snapshot_exists': paths2['feature_registry_snapshot'].exists(),
634     'all_features_raw_exists': paths2['all_features_raw'].exists(),
635     'all_features_stats_exists': paths2['all_features_stats'].exists(),
636     'stable_features_final_exists': paths2['stable_features_final'].exists(),
637     'feature_diagnostics_exists': paths2['feature_diagnostics'].exists(),
638     'step2_config_exists': paths2['step2_config'].exists(),
639     'stats_has_fdr': 'p_value_fdr' in stats_out.columns,
640     'stats_has_cliffs_delta': 'cliffs_delta' in stats_out.columns,
641     'stats_has_jsd': 'jsd' in stats_out.columns,
642     'registry_has_selected_final': 'selected_final' in final_registry.columns,
643     'stable_features_subset_consistent':
        set(final_stable_features['feature_name']).issubset(set(final_registry.loc[final_registry['sel
        'feature_name']])),
644 }
645
646 log_lines = [
647     'Step 2 finished.',
648     f'IMDb input: {imdb_path}',
649     f'GoEmotions binary input: {go_bin_path}',
650     f'GoEmotions multiclass input: {go_multi_path}',
651     "Effect size: Cliff's delta",
652     f"Continuous JSD bins: global edges with {cfg2['jsd_continuous_bins']} bins",
653     'Predictive fitting boundary: GoEmotions train only',
654     'FDR is retained as a diagnostic warning, not a hard exclusion rule',
655     f"Threshold mode: {stats_df['threshold_mode'].iloc[0] if len(stats_df) else 'NA'}",
656     f"KS cutoff used: {stats_df['ks_cutoff_used'].iloc[0] if len(stats_df) else 'NA'}",
657     f"JSD cutoff used: {stats_df['jsd_cutoff_used'].iloc[0] if len(stats_df) else
        'NA'}",
658     f"Cliff cutoff used: {stats_df['cliff_cutoff_used'].iloc[0] if len(stats_df) else
        'NA'}",
659     f"Stable candidate count: {int(stats_df['is_stable_candidate'].sum())}",
660     f"Final selected feature count: {int(stats_df['selected_final'].sum())}",
661     f'Output folder: {step2_dir}',
662     f"feature_registry: {paths2['feature_registry']}",
663     f"feature_registry_snapshot: {paths2['feature_registry_snapshot']}",
664     f"all_features_raw: {paths2['all_features_raw']}",
665     f"all_features_stats: {paths2['all_features_stats']}",
666     f"stable_features_final: {paths2['stable_features_final']}",
667     f"feature_diagnostics: {paths2['feature_diagnostics']}",

```

```

668         f'validation_comparison_rows: {len(compare_df)}',
669     ]
670
671     with open(paths2['step2_log'], 'w', encoding='utf-8') as f:
672         f.write('\n'.join(log_lines))
673
674     if not all(checks.values()):
675         failed = [k for k, v in checks.items() if not v]
676         raise ValueError(f'Step 2 final checks failed: {failed}')
677
678     pd.Series(checks)

```

Listing 2. *step2_features_final.ipynb*

step3_model.ipynb

```

1  # Run this once if needed.
2  %pip install -q numpy pandas scikit-learn torch transformers accelerate
3
4  from pathlib import Path
5  import json
6  import gc
7  import random
8  import warnings
9
10 import numpy as np
11 import pandas as pd
12
13 from sklearn.feature_extraction.text import TfidfVectorizer
14 from sklearn.linear_model import LogisticRegression
15 from sklearn.metrics import accuracy_score, f1_score, precision_score, recall_score,
16     roc_auc_score
17 from sklearn.model_selection import ParameterGrid, StratifiedKFold
18 from sklearn.pipeline import Pipeline
19 from sklearn.preprocessing import StandardScaler
20
21 warnings.filterwarnings('ignore')
22
23 root = Path.cwd()
24 data_dir = root / 'data'
25 out_dir = root / 'outputs'
26 step2_dir = out_dir / 'step2_features'
27 step3_dir = out_dir / 'step3_models'
28 models_dir = step3_dir / 'models'
29 preds_dir = step3_dir / 'predictions'
30 reports_dir = step3_dir / 'reports'
31
32 for p in [data_dir, out_dir, step3_dir, models_dir, preds_dir, reports_dir]:
33     p.mkdir(parents=True, exist_ok=True)
34
35 cfg3 = {
36     'seed': 42,
37     'stable_features': [
38         'exclaim_cnt',
39         'exclaim_per_100w',
40         'downtoner_cnt',
41         'downtoner_per_100w',
42     ],
43     'high_conf_threshold': 0.90,
44     'oof_n_splits': 5,
45     'oof_cv_type': 'StratifiedKFold',
46     'early_fusion_pooling': 'cls',
47     'residual_meta_model': 'logistic_regression',
48     'text_model_name': 'distilbert-base-uncased',
49     'text_max_length': 96,
50     'text_batch_size': 4,

```

```

50     'text_learning_rate': 2e-5,
51     'text_num_epochs': 1,
52     'text_weight_decay': 0.01,
53     'text_warmup_ratio': 0.0,
54     'tfidf_param_grid': {
55         'tfidf_ngram_range': [(1, 1), (1, 2)],
56         'tfidf_max_features': [20000, 40000],
57         'clf_C': [0.5, 1.0, 2.0],
58     },
59     'stable_lr_param_grid': {
60         'C': [0.1, 1.0, 5.0],
61     },
62     'residual_lr_C': 1.0,
63 }
64
65 with open(step3_dir / 'step3_config.json', 'w', encoding='utf-8') as f:
66     json.dump(cfg3, f, ensure_ascii=False, indent=2)
67
68 random.seed(cfg3['seed'])
69 np.random.seed(cfg3['seed'])
70
71 print('Current working directory:', root.resolve())
72 print('data_dir:', data_dir)
73 print('step2_dir:', step2_dir)
74 print('step3_dir:', step3_dir)
75 print(json.dumps(cfg3, ensure_ascii=False, indent=2))
76
77 from pathlib import Path
78
79 if 'root' not in globals():
80     root = Path.cwd()
81 if 'data_dir' not in globals():
82     data_dir = root / 'data'
83 if 'out_dir' not in globals():
84     out_dir = root / 'outputs'
85 if 'step2_dir' not in globals():
86     step2_dir = out_dir / 'step2_features'
87 if 'step3_dir' not in globals():
88     step3_dir = out_dir / 'step3_models'
89 if 'models_dir' not in globals():
90     models_dir = step3_dir / 'models'
91 if 'preds_dir' not in globals():
92     preds_dir = step3_dir / 'predictions'
93 if 'reports_dir' not in globals():
94     reports_dir = step3_dir / 'reports'
95 for p in [data_dir, out_dir, step2_dir, step3_dir, models_dir, preds_dir, reports_dir]:
96     p.mkdir(parents=True, exist_ok=True)
97
98 if 'cfg3' not in globals():
99     cfg3 = {
100         'seed': 42,
101         'stable_features': [
102             'exclaim_cnt',
103             'exclaim_per_100w',
104             'downtoner_cnt',
105             'downtoner_per_100w',
106         ],
107         'high_conf_threshold': 0.90,
108         'oof_n_splits': 5,
109         'oof_cv_type': 'StratifiedKFold',
110         'early_fusion_pooling': 'cls',
111         'residual_meta_model': 'logistic_regression',
112         'text_model_name': 'distilbert-base-uncased',
113         'text_max_length': 128,
114         'text_batch_size': 16,
115         'text_learning_rate': 2e-5,
116         'text_num_epochs': 2,
117         'text_weight_decay': 0.01,

```

```

118         'text_warmup_ratio': 0.1,
119         'tfidf_param_grid': {
120             'tfidf__ngram_range': [(1, 1), (1, 2)],
121             'tfidf__max_features': [20000, 40000],
122             'clf__C': [0.5, 1.0, 2.0],
123         },
124         'stable_lr_param_grid': {
125             'C': [0.1, 1.0, 5.0],
126         },
127         'residual_lr_C': 1.0,
128     }
129
130
131 def resolve_existing_path(candidates):
132     for fp in candidates:
133         if fp.exists():
134             return fp
135     raise FileNotFoundError('Missing required file. Checked: ' + ', '.join(str(x) for
136                                     x in candidates))
137
138 go_path = resolve_existing_path([
139     data_dir / 'clean_goemotions_binary.csv',
140     data_dir / 'gobin.csv',
141 ])
142
143 imdb_path = resolve_existing_path([
144     data_dir / 'clean_imdb_binary.csv',
145     data_dir / 'imdb.csv',
146 ])
147
148 stable_path = resolve_existing_path([
149     step2_dir / 'stable_features_final.csv',
150 ])
151
152 go_df = pd.read_csv(go_path)
153 imdb_df = pd.read_csv(imdb_path)
154 stable_df = pd.read_csv(stable_path)
155
156 if 'feature_name' not in stable_df.columns:
157     raise ValueError('stable_features_final.csv must include a feature_name column.')
158
159 stable_features_from_file = stable_df['feature_name'].dropna().astype(str).tolist()
160 missing_frozen = [c for c in cfg3['stable_features'] if c not in
161                   stable_features_from_file]
162 if missing_frozen:
163     raise ValueError(f'Frozen stable features missing from stable_features_final.csv:
164                       {missing_frozen}')
165
166
167 def ensure_basic_text_cols(df):
168     df = df.copy()
169     if 'text' not in df.columns:
170         raise ValueError('Input dataframe must include a text column.')
171     df['text'] = df['text'].astype(str)
172     if 'n_words' not in df.columns:
173         if 'nwords' in df.columns:
174             df['n_words'] = df['nwords']
175         else:
176             df['n_words'] = df['text'].str.split().apply(len)
177     return df
178
179 def add_missing_stable_features(df):
180     df = ensure_basic_text_cols(df)
181     downtoners = {
182         'slightly', 'somewhat', 'rather', 'fairly', 'barely',
183         'hardly', 'partly', 'partially', 'relatively', 'moderately',

```

```

183         'kind of', 'kinda', 'sort of', 'a bit', 'a little'
184     }
185
186     if 'exclaim_cnt' not in df.columns:
187         if 'exclaimcnt' in df.columns:
188             df['exclaim_cnt'] = df['exclaimcnt']
189         else:
190             df['exclaim_cnt'] = df['text'].str.count('!')
191
192     if 'exclaim_per_100w' not in df.columns:
193         denom = df['n_words'].replace(0, np.nan)
194         df['exclaim_per_100w'] = (df['exclaim_cnt'] / denom) * 100.0
195         df['exclaim_per_100w'] = df['exclaim_per_100w'].fillna(0.0)
196
197     if 'downtoner_cnt' not in df.columns:
198         text_lower = df['text'].str.lower()
199
200         def count_downtoners(x):
201             total = 0
202             for term in downtoners:
203                 total += x.count(term)
204             return total
205
206         df['downtoner_cnt'] = text_lower.apply(count_downtoners)
207
208     if 'downtoner_per_100w' not in df.columns:
209         denom = df['n_words'].replace(0, np.nan)
210         df['downtoner_per_100w'] = (df['downtoner_cnt'] / denom) * 100.0
211         df['downtoner_per_100w'] = df['downtoner_per_100w'].fillna(0.0)
212
213     return df
214
215
216 go_df = add_missing_stable_features(go_df)
217 imdb_df = add_missing_stable_features(imdb_df)
218
219 required_cols = ['id', 'text', 'label', 'split'] + cfg3['stable_features']
220 for name, df in [('go', go_df), ('imdb', imdb_df)]:
221     missing = [c for c in required_cols if c not in df.columns]
222     if missing:
223         raise ValueError(f'Missing columns in {name} dataframe: {missing}')
224
225 go_train = go_df[go_df['split'].astype(str).str.lower() == 'train'].copy()
226 go_valid = go_df[go_df['split'].astype(str).str.lower() == 'validation'].copy()
227 imdb_test = imdb_df[imdb_df['split'].astype(str).str.lower() == 'test'].copy()
228
229 for df in [go_train, go_valid, imdb_test]:
230     df['text'] = df['text'].astype(str)
231     df['label'] = df['label'].astype(int)
232
233 optional_cols = [c for c in ['lengthbin', 'length_bin'] if c in go_df.columns or c in
234                  imdb_df.columns]
235 keep_cols = ['id', 'text', 'label', 'split'] + optional_cols + cfg3['stable_features']
236
237 go_train = go_train[[c for c in keep_cols if c in go_train.columns]].copy()
238 go_valid = go_valid[[c for c in keep_cols if c in go_valid.columns]].copy()
239 imdb_test = imdb_test[[c for c in keep_cols if c in imdb_test.columns]].copy()
240
241 print('go_path:', go_path)
242 print('imdb_path:', imdb_path)
243 print('stable_path:', stable_path)
244 print('go_train:', go_train.shape)
245 print('go_valid:', go_valid.shape)
246 print('imdb_test:', imdb_test.shape)
247 print(go_train[cfg3['stable_features']].head())
248
249 def compute_metrics(y_true, y_pred, y_prob):
250     metrics = {

```

```

250         'f1': float(f1_score(y_true, y_pred, zero_division=0)),
251         'accuracy': float(accuracy_score(y_true, y_pred)),
252         'precision': float(precision_score(y_true, y_pred, zero_division=0)),
253         'recall': float(recall_score(y_true, y_pred, zero_division=0)),
254     }
255     try:
256         metrics['roc_auc'] = float(roc_auc_score(y_true, y_prob))
257     except Exception:
258         metrics['roc_auc'] = np.nan
259     return metrics
260
261
262 def prediction_frame(df, model_name, y_prob, threshold=0.5):
263     y_prob = np.asarray(y_prob, dtype=float)
264     y_pred = (y_prob >= threshold).astype(int)
265     out = df[['id', 'text', 'label']].copy()
266     out['model_name'] = model_name
267     out['pred_prob'] = y_prob
268     out['pred_label'] = y_pred
269     for c in cfg3['stable_features']:
270         out[c] = df[c].to_numpy()
271     for c in ['lengthbin', 'length_bin']:
272         if c in df.columns:
273             out[c] = df[c].astype(str).to_numpy()
274     return out
275
276
277 def append_result(rows, split_name, model_name, metric_dict, extra=None):
278     row = {'split_name': split_name, 'model_name': model_name}
279     row.update(metric_dict)
280     if extra:
281         row.update(extra)
282     rows.append(row)
283
284 results_rows = []
285
286 X_train_text = go_train['text'].astype(str).tolist()
287 y_train = go_train['label'].astype(int).to_numpy()
288 X_valid_text = go_valid['text'].astype(str).tolist()
289 y_valid = go_valid['label'].astype(int).to_numpy()
290 X_test_text = imdb_test['text'].astype(str).tolist()
291 y_test = imdb_test['label'].astype(int).to_numpy()
292
293 best_tfidf = None
294 best_valid_f1 = -1.0
295 best_tfidf_params = None
296
297 for params in ParameterGrid(cfg3['tfidf_param_grid']):
298     pipe = Pipeline([
299         ('tfidf', TfidfVectorizer(lowercase=True, strip_accents='unicode')),
300         ('clf', LogisticRegression(max_iter=2000, random_state=cfg3['seed']))
301     ])
302     pipe.set_params(**params)
303     pipe.fit(X_train_text, y_train)
304     valid_prob = pipe.predict_proba(X_valid_text)[: , 1]
305     valid_pred = (valid_prob >= 0.5).astype(int)
306     valid_f1 = f1_score(y_valid, valid_pred, zero_division=0)
307     if valid_f1 > best_valid_f1:
308         best_valid_f1 = valid_f1
309         best_tfidf = pipe
310         best_tfidf_params = params
311
312 val_prob = best_tfidf.predict_proba(X_valid_text)[: , 1]
313 test_prob = best_tfidf.predict_proba(X_test_text)[: , 1]
314
315 val_metrics = compute_metrics(y_valid, (val_prob >= 0.5).astype(int), val_prob)
316 test_metrics = compute_metrics(y_test, (test_prob >= 0.5).astype(int), test_prob)

```

```

317 append_result(results_rows, 'go_validation', 'L1_TFIDF_LR', val_metrics,
    {'best_params': json.dumps(best_tfidf_params)})
318 append_result(results_rows, 'imdb_test', 'L1_TFIDF_LR', test_metrics, {'best_params':
    json.dumps(best_tfidf_params)})
319
320 prediction_frame(go_valid, 'L1_TFIDF_LR', val_prob).to_csv(preds_dir /
    'l1_tfidf_lr_go_validation_preds.csv', index=False, encoding='utf-8-sig')
321 prediction_frame(imdb_test, 'L1_TFIDF_LR', test_prob).to_csv(preds_dir /
    'l1_tfidf_lr_imdb_test_preds.csv', index=False, encoding='utf-8-sig')
322
323 X_train_stable = go_train[cfg3['stable_features']].copy()
324 X_valid_stable = go_valid[cfg3['stable_features']].copy()
325 X_test_stable = imdb_test[cfg3['stable_features']].copy()
326
327 best_stable = None
328 best_valid_f1 = -1.0
329 best_stable_params = None
330
331 for params in ParameterGrid(cfg3['stable_lr_param_grid']):
332     pipe = Pipeline([
333         ('scaler', StandardScaler()),
334         ('clf', LogisticRegression(max_iter=2000, random_state=cfg3['seed'],
            C=params['C']))
335     ])
336     pipe.fit(X_train_stable, y_train)
337     valid_prob = pipe.predict_proba(X_valid_stable)[: , 1]
338     valid_pred = (valid_prob >= 0.5).astype(int)
339     valid_f1 = f1_score(y_valid, valid_pred, zero_division=0)
340     if valid_f1 > best_valid_f1:
341         best_valid_f1 = valid_f1
342         best_stable = pipe
343         best_stable_params = params
344
345 val_prob = best_stable.predict_proba(X_valid_stable)[: , 1]
346 test_prob = best_stable.predict_proba(X_test_stable)[: , 1]
347
348 val_metrics = compute_metrics(y_valid, (val_prob >= 0.5).astype(int), val_prob)
349 test_metrics = compute_metrics(y_test, (test_prob >= 0.5).astype(int), test_prob)
350 append_result(results_rows, 'go_validation', 'L3_STABLE_ONLY', val_metrics,
    {'best_params': json.dumps(best_stable_params)})
351 append_result(results_rows, 'imdb_test', 'L3_STABLE_ONLY', test_metrics,
    {'best_params': json.dumps(best_stable_params)})
352
353 prediction_frame(go_valid, 'L3_STABLE_ONLY', val_prob).to_csv(preds_dir /
    'l3_stable_only_go_validation_preds.csv', index=False, encoding='utf-8-sig')
354 prediction_frame(imdb_test, 'L3_STABLE_ONLY', test_prob).to_csv(preds_dir /
    'l3_stable_only_imdb_test_preds.csv', index=False, encoding='utf-8-sig')
355
356 import os
357 os.environ["CUDA_LAUNCH_BLOCKING"] = "1"
358
359 import torch
360 from torch import nn
361 from torch.utils.data import Dataset
362 from transformers import AutoTokenizer, AutoModel, AutoModelForSequenceClassification,
    Trainer, TrainingArguments, EarlyStoppingCallback
363
364 DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'
365 print('DEVICE:', DEVICE)
366
367 import inspect
368
369
370 class TextDataset(Dataset):
371     def __init__(self, texts, labels, tokenizer, max_length):
372         self.texts = list(texts)
373         self.labels = None if labels is None else list(labels)
374         self.tokenizer = tokenizer

```

```

375         self.max_length = max_length
376
377     def __len__(self):
378         return len(self.texts)
379
380     def __getitem__(self, idx):
381         enc = self.tokenizer(
382             self.texts[idx],
383             truncation=True,
384             padding='max_length',
385             max_length=self.max_length,
386             return_tensors='pt'
387         )
388         item = {k: v.squeeze(0) for k, v in enc.items()}
389         if self.labels is not None:
390             item['labels'] = torch.tensor(int(self.labels[idx]), dtype=torch.long)
391         return item
392
393
394 def trainer_compute_metrics(eval_pred):
395     logits, labels = eval_pred
396     probs = torch.softmax(torch.tensor(logits), dim=1)[: , 1].numpy()
397     preds = (probs >= 0.5).astype(int)
398     return compute_metrics(labels, preds, probs)
399
400
401 def build_training_arguments(output_dir):
402     sig = inspect.signature(TrainingArguments.__init__)
403     valid_keys = set(sig.parameters.keys())
404
405     arg_map = {
406         'output_dir': str(output_dir),
407         'learning_rate': cfg3['text_learning_rate'],
408         'per_device_train_batch_size': cfg3['text_batch_size'],
409         'per_device_eval_batch_size': cfg3['text_batch_size'],
410         'num_train_epochs': cfg3['text_num_epochs'],
411         'weight_decay': cfg3['text_weight_decay'],
412         'warmup_ratio': cfg3['text_warmup_ratio'],
413         'seed': cfg3['seed'],
414         'report_to': 'none',
415         'save_strategy': 'epoch',
416         'logging_strategy': 'steps',
417         'logging_steps': 100,
418         'load_best_model_at_end': True,
419         'metric_for_best_model': 'f1',
420         'greater_is_better': True,
421     }
422
423     if 'overwrite_output_dir' in valid_keys:
424         arg_map['overwrite_output_dir'] = True
425
426     if 'evaluation_strategy' in valid_keys:
427         arg_map['evaluation_strategy'] = 'epoch'
428     elif 'eval_strategy' in valid_keys:
429         arg_map['eval_strategy'] = 'epoch'
430
431     filtered = {k: v for k, v in arg_map.items() if k in valid_keys}
432     return TrainingArguments(**filtered)
433
434
435 def train_text_only_model(train_df, valid_df, run_name):
436     tokenizer = AutoTokenizer.from_pretrained(cfg3['text_model_name'])
437     model = AutoModelForSequenceClassification.from_pretrained(
438         cfg3['text_model_name'],
439         num_labels=2
440     )
441
442     train_ds = TextDataset(

```

```

443         train_df['text'].astype(str).tolist(),
444         train_df['label'].astype(int).tolist(),
445         tokenizer,
446         cfg3['text_max_length']
447     )
448     valid_ds = TextDataset(
449         valid_df['text'].astype(str).tolist(),
450         valid_df['label'].astype(int).tolist(),
451         tokenizer,
452         cfg3['text_max_length']
453     )
454
455     args = build_training_arguments(models_dir / run_name)
456
457     trainer = Trainer(
458         model=model,
459         args=args,
460         train_dataset=train_ds,
461         eval_dataset=valid_ds,
462         compute_metrics=trainer_compute_metrics,
463     )
464
465     trainer.train()
466     return trainer, tokenizer
467
468
469 def predict_text_only(trainer, tokenizer, df):
470     ds = TextDataset(
471         df['text'].astype(str).tolist(),
472         df['label'].astype(int).tolist(),
473         tokenizer,
474         cfg3['text_max_length']
475     )
476     pred_output = trainer.predict(ds)
477     logits = pred_output.predictions
478     probs = torch.softmax(torch.tensor(logits), dim=1)[: , 1].numpy()
479     return probs
480
481 text_trainer, text_tokenizer = train_text_only_model(go_train, go_valid,
482     'l5_text_only_main')
483
484 val_prob = predict_text_only(text_trainer, text_tokenizer, go_valid)
485 test_prob = predict_text_only(text_trainer, text_tokenizer, imdb_test)
486
487 val_metrics = compute_metrics(y_valid, (val_prob >= 0.5).astype(int), val_prob)
488 test_metrics = compute_metrics(y_test, (test_prob >= 0.5).astype(int), test_prob)
489 append_result(results_rows, 'go_validation', 'L5-TEXT-ONLY', val_metrics)
490 append_result(results_rows, 'imdb_test', 'L5-TEXT-ONLY', test_metrics)
491
492 prediction_frame(go_valid, 'L5-TEXT-ONLY', val_prob).to_csv(preds_dir /
493     'l5_text_only_go_validation_preds.csv', index=False, encoding='utf-8-sig')
494 prediction_frame(imdb_test, 'L5-TEXT-ONLY', test_prob).to_csv(preds_dir /
495     'l5_text_only_imdb_test_preds.csv', index=False, encoding='utf-8-sig')
496
497 skf = StratifiedKFold(n_splits=cfg3['oof_n_splits'], shuffle=True,
498     random_state=cfg3['seed'])
499 oof_prob = np.zeros(len(go_train), dtype=float)
500 fold_rows = []
501
502 for fold_id, (tr_idx, va_idx) in enumerate(skf.split(go_train,
503     go_train['label'].astype(int)), start=1):
504     fold_train = go_train.iloc[tr_idx].reset_index(drop=True)
505     fold_valid = go_train.iloc[va_idx].reset_index(drop=True)
506
507     trainer, tokenizer = train_text_only_model(fold_train, fold_valid,
508         f'l5_text_only_oof_fold_{fold_id}')
509     fold_prob = predict_text_only(trainer, tokenizer, fold_valid)
510     oof_prob[va_idx] = fold_prob

```

```

505
506     fold_pred = (fold_prob >= 0.5).astype(int)
507     fold_metrics = compute_metrics(fold_valid['label'].astype(int).to_numpy(),
508                                     fold_pred, fold_prob)
509     fold_rows.append({'fold_id': fold_id, **fold_metrics})
510
511     del trainer
512     gc.collect()
513     if torch.cuda.is_available():
514         torch.cuda.empty_cache()
515
516 oof_df = go_train[['id', 'text', 'label'] + cfg3['stable_features']].copy()
517 oof_df['text_only_oof_prob'] = oof_prob
518 oof_df['text_only_oof_pred'] = (oof_prob >= 0.5).astype(int)
519 oof_df.to_csv(preds_dir / 'oof_predictions_go.csv', index=False, encoding='utf-8-sig')
520 pd.DataFrame(fold_rows).to_csv(reports_dir / 'oof_fold_metrics.csv', index=False,
521                                 encoding='utf-8-sig')
522
523 class FusionDataset(Dataset):
524     def __init__(self, df, tokenizer, max_length, feature_cols):
525         self.df = df.reset_index(drop=True).copy()
526         self.tokenizer = tokenizer
527         self.max_length = max_length
528         self.feature_cols = list(feature_cols)
529
530     def __len__(self):
531         return len(self.df)
532
533     def __getitem__(self, idx):
534         row = self.df.iloc[idx]
535
536         enc = self.tokenizer(
537             str(row['text']),
538             truncation=True,
539             padding='max_length',
540             max_length=self.max_length,
541             return_tensors='pt'
542         )
543
544         item = {k: v.squeeze(0) for k, v in enc.items()}
545
546         feat = row[self.feature_cols].astype(float).to_numpy(dtype=np.float32)
547         item['features'] = torch.tensor(feat, dtype=torch.float32)
548         item['labels'] = torch.tensor(int(row['label']), dtype=torch.long)
549
550         return item
551
552 class EarlyFusionModel(nn.Module):
553     def __init__(self, model_name, num_extra_features, dropout=0.1):
554         super().__init__()
555         self.encoder = AutoModel.from_pretrained(model_name)
556         hidden_size = self.encoder.config.hidden_size
557         self.dropout = nn.Dropout(dropout)
558         self.classifier = nn.Linear(hidden_size + num_extra_features, 2)
559
560     def forward(self, input_ids=None, attention_mask=None, features=None, labels=None):
561         outputs = self.encoder(input_ids=input_ids, attention_mask=attention_mask)
562
563         if hasattr(outputs, 'last_hidden_state'):
564             cls_vec = outputs.last_hidden_state[:, 0, :]
565         else:
566             cls_vec = outputs[0][:, 0, :]
567
568         if features is None:
569             raise ValueError('features is None in EarlyFusionModel.forward().')
570
571         features = features.to(device=cls_vec.device, dtype=cls_vec.dtype)

```

```

571         if features.dim() != 2:
572             raise ValueError(f'features must be 2D, got shape {tuple(features.shape)}')
573
574         if cls_vec.size(0) != features.size(0):
575             raise ValueError(
576                 f'batch mismatch: cls batch={cls_vec.size(0)}, features
577                     batch={features.size(0)}'
578             )
579
580         merged = torch.cat([self.dropout(cls_vec), features], dim=1)
581         logits = self.classifier(merged)
582
583         loss = None
584         if labels is not None:
585             labels = labels.to(logits.device)
586             loss = nn.CrossEntropyLoss()(logits, labels)
587
588         return {'loss': loss, 'logits': logits}
589
590
591 def train_early_fusion_model(train_df, valid_df, run_name):
592     tokenizer = AutoTokenizer.from_pretrained(cfg3['text_model_name'])
593     model = EarlyFusionModel(cfg3['text_model_name'], len(cfg3['stable_features']))
594
595     train_ds = FusionDataset(
596         train_df,
597         tokenizer,
598         cfg3['text_max_length'],
599         cfg3['stable_features']
600     )
601     valid_ds = FusionDataset(
602         valid_df,
603         tokenizer,
604         cfg3['text_max_length'],
605         cfg3['stable_features']
606     )
607
608     args = build_training_arguments(models_dir / run_name)
609
610     trainer = Trainer(
611         model=model,
612         args=args,
613         train_dataset=train_ds,
614         eval_dataset=valid_ds,
615         compute_metrics=trainer_compute_metrics,
616     )
617
618     trainer.train()
619     return trainer, tokenizer
620
621
622 def predict_early_fusion(trainer, tokenizer, df):
623     ds = FusionDataset(
624         df,
625         tokenizer,
626         cfg3['text_max_length'],
627         cfg3['stable_features']
628     )
629     pred_output = trainer.predict(ds)
630     logits = pred_output.predictions
631     probs = torch.softmax(torch.tensor(logits), dim=1)[: , 1].numpy()
632     return probs
633
634 early_trainer, early_tokenizer = train_early_fusion_model(
635     go_train,
636     go_valid,
637     'l5_early_fusion_main'

```

```

638 )
639
640 val_prob = predict_early_fusion(early_trainer, early_tokenizer, go_valid)
641 test_prob = predict_early_fusion(early_trainer, early_tokenizer, imdb_test)
642
643 val_metrics = compute_metrics(
644     go_valid['label'].astype(int).to_numpy(),
645     (val_prob >= 0.5).astype(int),
646     val_prob
647 )
648
649 test_metrics = compute_metrics(
650     imdb_test['label'].astype(int).to_numpy(),
651     (test_prob >= 0.5).astype(int),
652     test_prob
653 )
654 append_result(results_rows, 'go_validation', 'L5_EARLY_FUSION', val_metrics)
655 append_result(results_rows, 'imdb_test', 'L5_EARLY_FUSION', test_metrics)
656
657 prediction_frame(go_valid, 'L5_EARLY_FUSION', val_prob).to_csv(
658     preds_dir / 'l5_early_fusion_go_validation_preds.csv',
659     index=False,
660     encoding='utf-8-sig'
661 )
662 prediction_frame(imdb_test, 'L5_EARLY_FUSION', test_prob).to_csv(
663     preds_dir / 'l5_early_fusion_imdb_test_preds.csv',
664     index=False,
665     encoding='utf-8-sig'
666 )
667
668 try:
669     results_rows
670 except NameError:
671     results_rows = []
672
673 append_result
674 prediction_frame
675 compute_metrics
676
677 val_metrics = compute_metrics(
678     go_valid['label'].astype(int).to_numpy(),
679     (val_prob >= 0.5).astype(int),
680     val_prob
681 )
682
683 test_metrics = compute_metrics(
684     imdb_test['label'].astype(int).to_numpy(),
685     (test_prob >= 0.5).astype(int),
686     test_prob
687 )
688
689 append_result(results_rows, 'go_validation', 'L5_EARLY_FUSION', val_metrics)
690 append_result(results_rows, 'imdb_test', 'L5_EARLY_FUSION', test_metrics)
691
692 prediction_frame(go_valid, 'L5_EARLY_FUSION', val_prob).to_csv(
693     preds_dir / 'l5_early_fusion_go_validation_preds.csv',
694     index=False,
695     encoding='utf-8-sig'
696 )
697
698 prediction_frame(imdb_test, 'L5_EARLY_FUSION', test_prob).to_csv(
699     preds_dir / 'l5_early_fusion_imdb_test_preds.csv',
700     index=False,
701     encoding='utf-8-sig'
702 )
703
704 print('Validation metrics:', val_metrics)
705 print('Test metrics:', test_metrics)

```

```

706 print('results_rows length:', len(results_rows))
707 print(results_rows[-2:])
708
709 print((preds_dir / 'l5_early_fusion_go_validation_preds.csv').exists())
710 print((preds_dir / 'l5_early_fusion_imdb_test_preds.csv').exists())
711
712 results_df = pd.DataFrame(results_rows)
713 results_df.to_csv(
714     preds_dir / 'l5_early_fusion_results.csv',
715     index=False,
716     encoding='utf-8-sig'
717 )
718
719 print((preds_dir / 'l5_early_fusion_results.csv').exists())
720 results_df
721
722 results_df = pd.DataFrame(results_rows)
723 results_df = results_df.drop_duplicates(
724     subset=['split_name', 'model_name'],
725     keep='last'
726 ).reset_index(drop=True)
727
728 results_rows = results_df.to_dict('records')
729
730 print('results_rows length =', len(results_rows))
731 results_df
732
733 results_df[['split_name', 'model_name']]
734
735 oof_df = pd.read_csv(preds_dir / 'oof_predictions_go.csv')
736 text_val_df = pd.read_csv(preds_dir / 'l5_text_only_go_validation_preds.csv')
737 text_test_df = pd.read_csv(preds_dir / 'l5_text_only_imdb_test_preds.csv')
738
739 residual_feature_cols = ['text_only_oof_prob'] + cfg3['stable_features']
740 residual_train = oof_df[['label'] + residual_feature_cols].copy()
741
742 residual_clf = Pipeline([
743     ('scaler', StandardScaler()),
744     ('clf', LogisticRegression(max_iter=2000, random_state=cfg3['seed'],
745         C=cfg3['residual_lr_C']))
746 ])
747 residual_clf.fit(residual_train[residual_feature_cols],
748     residual_train['label'].astype(int))
749
750 val_residual_input = go_valid[cfg3['stable_features']].copy()
751 val_residual_input.insert(0, 'text_only_oof_prob',
752     text_val_df['pred_prob'].astype(float).to_numpy())
753 val_prob = residual_clf.predict_proba(val_residual_input[residual_feature_cols])[:, 1]
754
755 imdb_residual_input = imdb_test[cfg3['stable_features']].copy()
756 imdb_residual_input.insert(0, 'text_only_oof_prob',
757     text_test_df['pred_prob'].astype(float).to_numpy())
758 test_prob = residual_clf.predict_proba(imdb_residual_input[residual_feature_cols])[:,
759     1]
760
761 val_metrics = compute_metrics(y_valid, (val_prob >= 0.5).astype(int), val_prob)
762 test_metrics = compute_metrics(y_test, (test_prob >= 0.5).astype(int), test_prob)
763 append_result(results_rows, 'go_validation', 'L5_RESIDUAL_FUSION', val_metrics)
764 append_result(results_rows, 'imdb_test', 'L5_RESIDUAL_FUSION', test_metrics)
765
766 prediction_frame(go_valid, 'L5_RESIDUAL_FUSION', val_prob).to_csv(preds_dir /
767     'l5_residual_fusion_go_validation_preds.csv', index=False, encoding='utf-8-sig')
768 prediction_frame(imdb_test, 'L5_RESIDUAL_FUSION', test_prob).to_csv(preds_dir /
769     'l5_residual_fusion_imdb_test_preds.csv', index=False, encoding='utf-8-sig')
770
771 def compute_ecr_tables(text_df, fusion_df, dataset_name):
772     merged = text_df[['id', 'text', 'label', 'pred_prob',
773         'pred_label']].copy().rename(columns={

```

```

766         'pred_prob': 'text_only_prob',
767         'pred_label': 'text_only_pred',
768     })
769     fused = fusion_df[['id', 'pred_prob', 'pred_label']].copy().rename(columns={
770         'pred_prob': 'fusion_prob',
771         'pred_label': 'fusion_pred',
772     })
773     merged = merged.merge(fused, on='id', how='left')
774
775     base_wrong = merged['text_only_pred'].astype(int) != merged['label'].astype(int)
776     fusion_correct = merged['fusion_pred'].astype(int) == merged['label'].astype(int)
777     corrected = base_wrong & fusion_correct
778
779     text_conf = np.maximum(merged['text_only_prob'].astype(float), 1.0 -
780         merged['text_only_prob'].astype(float))
781     high_conf_wrong = base_wrong & (text_conf >= cfg3['high_conf_threshold'])
782     hc_corrected = high_conf_wrong & fusion_correct
783
784     denom_ecr = int(base_wrong.sum())
785     denom_hc = int(high_conf_wrong.sum())
786
787     ecr = float(corrected.sum() / denom_ecr) if denom_ecr > 0 else np.nan
788     hc_ecr = float(hc_corrected.sum() / denom_hc) if denom_hc > 0 else np.nan
789
790     merged['was_text_only_wrong'] = base_wrong
791     merged['was_corrected'] = corrected
792     merged['text_only_conf'] = text_conf
793     merged['was_high_confidence_error'] = high_conf_wrong
794     merged['was_high_confidence_corrected'] = hc_corrected
795     merged['dataset_name'] = dataset_name
796
797     summary = {
798         'dataset_name': dataset_name,
799         'text_only_errors': denom_ecr,
800         'corrected_errors': int(corrected.sum()),
801         'ecr': ecr,
802         'text_only_high_conf_errors': denom_hc,
803         'high_conf_corrected_errors': int(hc_corrected.sum()),
804         'hc_ecr': hc_ecr,
805     }
806     return merged, summary
807
808 val_text = pd.read_csv(preds_dir / 'l5_text_only_go_validation_preds.csv')
809 val_fusion = pd.read_csv(preds_dir / 'l5_residual_fusion_go_validation_preds.csv')
810 val_cases, val_ecr = compute_ecr_tables(val_text, val_fusion, 'go_validation')
811
812 imdb_text = pd.read_csv(preds_dir / 'l5_text_only_imdb_test_preds.csv')
813 imdb_fusion = pd.read_csv(preds_dir / 'l5_residual_fusion_imdb_test_preds.csv')
814 imdb_cases, imdb_ecr = compute_ecr_tables(imdb_text, imdb_fusion, 'imdb_test')
815
816 feature_lookup = pd.concat([
817     go_valid[['id'] + cfg3['stable_features'] + [c for c in ['lengthbin',
818         'length_bin'] if c in go_valid.columns]],
819     imdb_test[['id'] + cfg3['stable_features'] + [c for c in ['lengthbin',
820         'length_bin'] if c in imdb_test.columns]],
821 ], ignore_index=True).drop_duplicates('id')
822
823 error_cases = pd.concat([val_cases, imdb_cases], ignore_index=True)
824 error_cases = error_cases.merge(feature_lookup, on='id', how='left')
825 error_cases.to_csv(reports_dir / 'error_case_analysis.csv', index=False,
826     encoding='utf-8-sig')
827 pd.DataFrame([val_ecr, imdb_ecr]).to_csv(reports_dir / 'ecr_summary.csv', index=False,
828     encoding='utf-8-sig')
829
830 results_df = pd.DataFrame(results_rows)
831 results_df.to_csv(reports_dir / 'results_summary.csv', index=False,
832     encoding='utf-8-sig')

```

```

828 log_lines = [
829     'Step 3 finished.',
830     f'GoEmotions input: {go_path}',
831     f'IMDb input: {imdb_path}',
832     f'Stable feature input: {stable_path}',
833     f'Stable features: {cfg3["stable_features"]}',
834     f'Text backbone: {cfg3["text_model_name"]}',
835     f'OOF folds: {cfg3["oof_n_splits"]}',
836     f'OOF CV type: {cfg3["oof_cv_type"]}',
837     f'High-confidence threshold: {cfg3["high_conf_threshold"]}',
838     f'Output folder: {step3_dir}',
839     f'Results summary: {reports_dir / "results_summary.csv"}',
840     f'ECR summary: {reports_dir / "ecr_summary.csv"}',
841     f'Error case analysis: {reports_dir / "error_case_analysis.csv"}',
842 ]
843 with open(step3_dir / 'step3_log.txt', 'w', encoding='utf-8') as f:
844     f.write("\n".join(log_lines))
845
846 try:
847     print((outdir / 'results_summary.csv').exists())
848     print((outdir / 'ecr_summary.csv').exists())
849 except NameError:
850     from pathlib import Path
851     print(Path('results_summary.csv').exists())
852     print(Path('ecr_summary.csv').exists())
853
854 results_rows = []
855
856 reports_dir = preds_dir.parent / 'reports'
857 results_summary = pd.read_csv(reports_dir / 'results_summary.csv')
858 ecr_summary = pd.read_csv(reports_dir / 'ecr_summary.csv')
859
860 results_summary
861 ecr_summary

```

Listing 3. *step3_{model}.ipynb*

step4_{report}.ipynb

```

1  from pathlib import Path
2  import json
3  import pandas as pd
4  import numpy as np
5
6  try:
7      import plotly.express as px
8      import plotly.graph_objects as go
9      PLOTLY_OK = True
10 except Exception:
11     PLOTLY_OK = False
12
13 root = Path(r'C:\Users\cdex1\Desktop\          \90\ass2\  2  ')
14 reports_dir = root / 'outputs' / 'step3_models' / 'reports'
15 preds_dir = root / 'outputs' / 'step3_models' / 'predictions'
16 step4_dir = root / 'outputs' / 'step4_report'
17 step4_dir.mkdir(parents=True, exist_ok=True)
18
19 results_summary = pd.read_csv(reports_dir / 'results_summary.csv')
20 ecr_summary = pd.read_csv(reports_dir / 'ecr_summary.csv')
21 error_case_analysis = pd.read_csv(reports_dir / 'error_case_analysis.csv')
22
23 config_path = root / 'outputs' / 'data_prep' / 'config.json'
24 config_data = json.loads(config_path.read_text(encoding='utf-8')) if
    config_path.exists() else {}
25 seed = config_data.get('cfg', {}).get('seed', 42)
26

```

```

27 required = pd.DataFrame([
28     {'artifact': 'results_summary.csv', 'exists': (reports_dir /
        'results_summary.csv').exists(), 'path': str(reports_dir /
        'results_summary.csv')},
29     {'artifact': 'ecr_summary.csv', 'exists': (reports_dir /
        'ecr_summary.csv').exists(), 'path': str(reports_dir / 'ecr_summary.csv')},
30     {'artifact': 'error_case_analysis.csv', 'exists': (reports_dir /
        'error_case_analysis.csv').exists(), 'path': str(reports_dir /
        'error_case_analysis.csv')},
31     {'artifact': 'l5_early_fusion_results.csv', 'exists': (preds_dir /
        'l5_early_fusion_results.csv').exists(), 'path': str(preds_dir /
        'l5_early_fusion_results.csv')},
32 ])
33 required
34
35 model_order = ['L1_TFIDF_LR', 'L3_STABLE_ONLY', 'L5_TEXT_ONLY', 'L5_EARLY_FUSION',
    'L5_RESIDUAL_FUSION']
36 label_map = {
37     'L1_TFIDF_LR': 'L1 TF-IDF',
38     'L3_STABLE_ONLY': 'L3 Stable-only',
39     'L5_TEXT_ONLY': 'L5 Text-only',
40     'L5_EARLY_FUSION': 'L5 Early',
41     'L5_RESIDUAL_FUSION': 'L5 Residual'
42 }
43 plot_df = results_summary[results_summary['model_name'].isin(model_order)].copy()
44 plot_df['model_short'] = plot_df['model_name'].map(label_map)
45 plot_df['dataset_short'] = plot_df['split_name'].map({'go_validation': 'GoEmotions
    Validation', 'imdb_test': 'IMDb Test'})
46 plot_df = plot_df.sort_values(['model_name', 'split_name'])
47 plot_df[['model_short', 'dataset_short', 'f1']]
48
49 if PLOTLY_OK:
50     fig = px.bar(
51         plot_df,
52         x='model_short',
53         y='f1',
54         color='dataset_short',
55         barmode='group',
56         title='Domain Shift Penalty: Validation vs Test F1',
57         labels={'model_short': 'Model', 'f1': 'F1', 'dataset_short': 'Split'},
58         color_discrete_sequence=['#2E86AB', '#F18F01']
59     )
60     fig.update_layout(template='plotly_white', legend_title_text='Split', height=540,
        width=980)
61     fig.write_html(step4_dir / 'domain_shift_penalty.html')
62     print(step4_dir / 'domain_shift_penalty.html')
63 else:
64     print('plotly unavailable; skipping domain-shift HTML chart')
65
66 if {'text_only_prob', 'fusion_prob'}.issubset(error_case_analysis.columns) and
    PLOTLY_OK:
67     ec = error_case_analysis.copy()
68     fig = px.scatter(
69         ec,
70         x='text_only_prob',
71         y='fusion_prob',
72         color='dataset_name' if 'dataset_name' in ec.columns else None,
73         hover_data=[c for c in ['text', 'note', 'exclaim_cnt', 'downtoner_cnt'] if c
            in ec.columns],
74         title='Residual Correction: Text-only Probability vs Fusion Probability',
75         labels={'text_only_prob': 'Text-only probability', 'fusion_prob': 'Fusion
            probability'})
76
77     fig.add_hline(y=0.5, line_dash='dash', line_color='gray')
78     fig.add_vline(x=0.5, line_dash='dash', line_color='gray')
79     fig.update_layout(template='plotly_white', height=560, width=980)
80     fig.write_html(step4_dir / 'residual_correction_scatter.html')
81     print(step4_dir / 'residual_correction_scatter.html')

```

```

82 else:
83     print('scatter chart skipped because plotly is unavailable or required columns are
        missing')
84
85 ecr_summary.assign(seed=seed)
86
87 case_cols = [c for c in ['dataset_name', 'text', 'text_only_prob', 'fusion_prob',
        'label', 'pred_text_only', 'pred_fusion', 'exclaim_cnt', 'question_cnt',
        'downtoner_cnt', 'exclaim_per_100w', 'downtoner_per_100w', 'note'] if c in
        error_case_analysis.columns]
88 case_preview = error_case_analysis[case_cols].head(8).copy()
89 case_preview.to_csv(step4_dir / 'case_study_preview.csv', index=False,
        encoding='utf-8-sig')
90 case_preview
91
92 repro = pd.DataFrame([
93     {'item': 'seed', 'value': seed},
94     {'item': 'results_summary', 'value': str(reports_dir / 'results_summary.csv')},
95     {'item': 'ecr_summary', 'value': str(reports_dir / 'ecr_summary.csv')},
96     {'item': 'error_case_analysis', 'value': str(reports_dir /
        'error_case_analysis.csv')},
97     {'item': 'predictions_dir', 'value': str(preds_dir)},
98     {'item': 'reports_dir', 'value': str(reports_dir)},
99     {'item': 'step4_notebook', 'value': str(step4_dir / 'step4_report3.ipynb')},
100 ])
101 repro.to_csv(step4_dir / 'reproducibility_checklist.csv', index=False,
        encoding='utf-8-sig')
102 repro
103
104 report_md = (
105     '# Step 4 Results & Analysis\n\n'
106     '## Overall performance\n'
107     'The grouped comparison shows a clear domain shift penalty from GoEmotions
        validation to IMDb test. '
108     'L1 TF-IDF and L5 text-only perform strongly on validation but drop on IMDb, while
        stable-only features remain predictive but do not close the gap. '
109     'Early fusion and residual fusion improve local behavior more than global F1.\n\n'
110     '## Residual correction\n'
111     'Residual fusion corrects a small but meaningful subset of text-only errors. The
        ECR is low and HC-ECR is extremely small, '
112     'which suggests the stable features work as a local correction signal rather than
        a global replacement for text evidence.\n\n'
113     '## Case studies\n'
114     'Use 'case_study_preview.csv' to highlight concrete examples where punctuation
        density or downtoner patterns moved predictions across the 0.5 threshold.\n\n'
115     '## Reproducibility checklist\n'
116     f'- Seed: {seed}\n'
117     '- Raw and cleaned data: 'data/'\n'
118     '- Predictions: 'outputs/step3_models/predictions/'\n'
119     '- Reports: 'outputs/step3_models/reports/'\n'
120     '- Step 4 notebook: 'outputs/step4_report/step4_report3.ipynb'\n'
121 )
122 (step4_dir / 'report.md').write_text(report_md, encoding='utf-8')
123 print(report_md)

```

Listing 4. *step4_report.ipynb*

step5_dynamic_report.ipynb

```

1 %pip install plotly
2
3 from pathlib import Path
4 import pandas as pd
5 import numpy as np
6 import plotly.express as px
7 import plotly.graph_objects as go

```

```

8
9 root = Path(r'C:\Users\cdex1\Desktop\          \90\ass2\  2  ')
10 step3_dir = root / 'outputs' / 'step3_models' / 'reports'
11 step5_dir = root / 'outputs' / 'step5_reports'
12 step5_dir.mkdir(parents=True, exist_ok=True)
13
14 results_path = step3_dir / 'results_summary.csv'
15 ecr_path = step3_dir / 'ecr_summary.csv'
16 err_path = step3_dir / 'error_case_analysis.csv'
17 feature_stats_path = step3_dir / 'all_features_stats.csv'
18 stable_final_path = step3_dir / 'stable_features_final.csv'
19
20 checks = pd.DataFrame([
21     {'file': 'results_summary.csv', 'exists': results_path.exists(), 'path':
22         str(results_path)},
23     {'file': 'ecr_summary.csv', 'exists': ecr_path.exists(), 'path': str(ecr_path)},
24     {'file': 'error_case_analysis.csv', 'exists': err_path.exists(), 'path':
25         str(err_path)},
26     {'file': 'all_features_stats.csv', 'exists': feature_stats_path.exists(), 'path':
27         str(feature_stats_path)},
28     {'file': 'stable_features_final.csv', 'exists': stable_final_path.exists(),
29         'path': str(stable_final_path)},
30 ])
31 checks
32
33 results = pd.read_csv(results_path)
34 ecr = pd.read_csv(ecr_path)
35 err = pd.read_csv(err_path)
36 results.head()
37
38 model_order =
39     ['L1_TFIDF_LR', 'L3_STABLE_ONLY', 'L5_TEXT_ONLY', 'L5_EARLY_FUSION', 'L5_RESIDUAL_FUSION']
40 label_map = {'L1_TFIDF_LR': 'L1 TF-IDF', 'L3_STABLE_ONLY': 'L3
41     Stable-only', 'L5_TEXT_ONLY': 'L5 Text-only', 'L5_EARLY_FUSION': 'L5
42     Early', 'L5_RESIDUAL_FUSION': 'L5 Residual'}
43 plot_df = results[results['model_name'].isin(model_order)].copy()
44 plot_df['model_short'] = plot_df['model_name'].map(label_map)
45 plot_df['dataset_short'] = plot_df['split_name'].map({'go_validation': 'GoEmotions
46     Validation', 'imdb_test': 'IMDb Test'})
47 fig = px.bar(plot_df, x='model_short', y='f1', color='dataset_short', barmode='group',
48     title='Model Performance Summary',
49     labels={'model_short': 'Model', 'f1': 'F1', 'dataset_short': 'Split'})
50 fig.update_layout(template='plotly_white', height=520, width=980)
51 fig.write_html(step5_dir / 'figure_model_performance.html')
52 print(step5_dir / 'figure_model_performance.html')
53
54 ecr_df = ecr.copy()
55 fig = go.Figure()
56 fig.add_trace(go.Bar(x=ecr_df['dataset_name'], y=ecr_df['ecr'], name='ECR'))
57 fig.add_trace(go.Bar(x=ecr_df['dataset_name'], y=ecr_df['hc_ecr'], name='HC-ECR'))
58 fig.update_layout(barmode='group', template='plotly_white', title='Error Correction
59     Rates', height=500, width=900)
60 fig.write_html(step5_dir / 'figure_ecr.html')
61 print(step5_dir / 'figure_ecr.html')
62
63 if feature_stats_path.exists():
64     feat = pd.read_csv(feature_stats_path)
65     cols = [c for c in feat.columns if c.lower() in
66         ['feature_name', 'feature_group', 'jsd', 'stability_score', 'effect_size', 'selected_final']]
67     feat[cols].head().to_csv(step5_dir / 'table_feature_stability_preview.csv',
68         index=False, encoding='utf-8-sig')
69     fig = px.scatter(feat, x='jsd', y='stability_score' if 'stability_score' in
70         feat.columns else feat.columns[0], color='feature_group' if 'feature_group' in
71         feat.columns else None, title='Feature Stability vs Predictive Signal')
72     fig.update_layout(template='plotly_white', height=540, width=980)
73     fig.write_html(step5_dir / 'figure_feature_stability.html')
74     print(step5_dir / 'figure_feature_stability.html')
75

```

```

61 if stable_final_path.exists():
62     stable = pd.read_csv(stable_final_path)
63     stable.head().to_csv(step5_dir / 'table_top_stable_features.csv', index=False,
64                          encoding='utf-8-sig')
65
66 if err.shape[0] > 0:
67     preview = err.head(10).copy()
68     preview.to_csv(step5_dir / 'table_error_cases_preview.csv', index=False,
69                  encoding='utf-8-sig')
70     if {'text_only_prob', 'fusion_prob'}.issubset(err.columns):
71         fig = px.scatter(err, x='text_only_prob', y='fusion_prob', title='Text-only vs
72                               Fusion Probability', opacity=0.7)
73         fig.add_hline(y=0.5, line_dash='dash', line_color='gray')
74         fig.add_vline(x=0.5, line_dash='dash', line_color='gray')
75         fig.update_layout(template='plotly_white', height=560, width=980)
76         fig.write_html(step5_dir / 'figure_error_cases.html')
77         print(step5_dir / 'figure_error_cases.html')
78
79 summary = pd.DataFrame([
80     {'item': 'results_file', 'value': str(results_path)},
81     {'item': 'ecr_file', 'value': str(ecr_path)},
82     {'item': 'error_case_file', 'value': str(err_path)},
83     {'item': 'feature_stats_file', 'value': str(feature_stats_path)},
84     {'item': 'stable_features_file', 'value': str(stable_final_path)},
85 ])
86 summary

```

Listing 5. *step5dynamic_report.ipynb*

step5_inventory.ipynb

```

1 from pathlib import Path
2 import json
3 import pandas as pd
4 from datetime import datetime
5
6 root = Path(r'C:\Users\cdex1\Desktop\          \90\ass2\    2    ')
7 paths_to_scan = [
8     root / 'data',
9     root / 'outputs',
10 ]
11 inventory_dir = root / 'outputs' / 'step5_inventory'
12 inventory_dir.mkdir(parents=True, exist_ok=True)
13
14 rows = []
15 for base in paths_to_scan:
16     if not base.exists():
17         continue
18     for f in base.rglob('*'):
19         if f.is_file():
20             stat = f.stat()
21             rel = f.relative_to(root)
22             stage = rel.parts[1] if len(rel.parts) > 1 and rel.parts[0] in ['data',
23                                     'outputs'] else 'root'
24             ext = f.suffix.lower()
25             rows.append({
26                 'file_name': f.name,
27                 'file_path': str(f),
28                 'relative_path': str(rel),
29                 'folder': str(f.parent),
30                 'stage': stage,
31                 'extension': ext,
32                 'size_bytes': stat.st_size,
33                 'modified_time':
34                     datetime.fromtimestamp(stat.st_mtime).isoformat(timespec='seconds')
35             })
36
37 })

```

```

34 inv = pd.DataFrame(rows).sort_values(['stage','relative_path']).reset_index(drop=True)
35 inv.head(20)
36
37 descriptions = {
38     'prep.ipynb': 'Data-preparation notebook that downloads GoEmotions and IMDb,
39                 builds cleaned datasets, and writes profiling outputs.',
40     'config.json': 'Configuration file storing seeds, paths, and label-mapping rules
41                 for preprocessing.',
42     'results_summary.csv': 'Step 3 summary table with model-level performance metrics
43                 on validation and IMDb test splits.',
44     'ecr_summary.csv': 'Step 3 error-correction summary with ECR and HC-ECR for
45                 residual fusion analysis.',
46     'error_case_analysis.csv': 'Step 3 sample-level case analysis describing corrected
47                 errors and feature signals.',
48     'step4_report.ipynb': 'First Step 4 report notebook draft for performance
49                 visualization and narrative writing.',
50     'step4_report2.ipynb': 'Improved Step 4 notebook with report artifacts, case-study
51                 preview, and reproducibility checklist.',
52     'step4_report3.ipynb': 'Final Step 4 notebook with validated charts, case-study
53                 export, and report text generation.',
54     'report.md': 'Step 4 markdown summary text for direct reuse in the assignment
55                 write-up.',
56     'case_study_preview.csv': 'Compact preview of representative corrected examples
57                 used in Step 4 case studies.',
58     'reproducibility_checklist.csv': 'Reproducibility checklist listing seed, inputs,
59                 and key output paths.',
60     'domain_shift_penalty.html': 'Interactive grouped bar chart showing
61                 validation-versus-test F1 drop across models.',
62     'residual_correction_rates.html': 'Interactive bar chart showing ECR and HC-ECR
63                 rates.',
64     'residual_correction_scatter.html': 'Scatter plot of text-only probability versus
65                 fusion probability with 0.5 decision lines.',
66     'step4_report.html': 'If present, an exported HTML version or related report
67                 artifact for Step 4.',
68     'file_inventory.ipynb': 'Step 5 inventory notebook that scans project files and
69                 writes a file catalog.',
70 }
71 inv['description_short'] = inv['file_name'].map(descriptions).fillna('Project file
72 generated during data prep, modeling, analysis, or reporting.')
73 inv['used_in_report'] =
74     inv['file_name'].isin(['results_summary.csv','ecr_summary.csv','error_case_analysis.csv','report.m
75 False:'maybe'])
76 inv['file_type'] = inv['extension'].replace({'': 'no_extension'})
77 inv.head(20)
78
79 check_targets = [
80     'results_summary.csv',
81     'ecr_summary.csv',
82     'error_case_analysis.csv',
83     'step4_report3.ipynb',
84     'report.md',
85     'case_study_preview.csv',
86     'reproducibility_checklist.csv',
87 ]
88 checks = []
89 for t in check_targets:
90     match = inv[inv['file_name'] == t]
91     checks.append({
92         'file': t,
93         'exists': not match.empty,
94         'count': int(len(match)),
95         'paths': '; '.join(match['relative_path'].tolist()[:3]) if not match.empty
96         else ''
97     })
98 checks_df = pd.DataFrame(checks)
99 checks_df
100
101 inv.to_csv(inventory_dir / 'file_inventory.csv', index=False, encoding='utf-8-sig')

```

```

82 checks_df.to_csv(inventory_dir / 'inventory_checks.csv', index=False,
83                  encoding='utf-8-sig')
84 (inv['description_short'].notna().all(), (inventory_dir /
85     'file_inventory.csv').exists(), (inventory_dir / 'inventory_checks.csv').exists())
86 md_lines = ['# File Inventory', '', 'This file lists the main project outputs, their
87             paths, and brief descriptions.', '']
88 for stage, grp in inv.groupby('stage'):
89     md_lines.append(f'## {stage}')
90     for _, r in grp.iterrows():
91         md_lines.append(f"- **{r['file_name']}**      {r['relative_path']}.
92                        {r['description_short']}")
93     md_lines.append('')
94 (inventory_dir / 'file_inventory.md').write_text('\n'.join(md_lines), encoding='utf-8')
95 print('\n'.join(md_lines[:40]))
96
97 summary = pd.DataFrame([
98     {'item': 'total_files_scanned', 'value': len(inv)},
99     {'item': 'stages_found', 'value': ',
100      '.join(sorted(inv['stage'].dropna().unique()))},
101     {'item': 'inventory_csv', 'value': str(inventory_dir / 'file_inventory.csv')},
102     {'item': 'inventory_md', 'value': str(inventory_dir / 'file_inventory.md')},
103     {'item': 'inventory_checks', 'value': str(inventory_dir / 'inventory_checks.csv')},
104 ])
105 summary

```

Listing 6. `step5inventory.ipynb`

`step5staticreport.ipynb`

```

1  from pathlib import Path
2  import json
3  import pandas as pd
4  import numpy as np
5
6  root = Path(r'C:\Users\cdex1\Desktop\          \90\ass2\    2    ')
7  prep_dir = root / 'outputs' / 'data_prep'
8  out_dir = root / 'outputs' / 'step5_reports'
9  out_dir.mkdir(parents=True, exist_ok=True)
10
11  config_path = prep_dir / 'config.json'
12  profile_candidates = [prep_dir / 'profile.csv', prep_dir /
13                        'corpus_profile_report.csv', prep_dir / 'readinesscheck.csv']
14  log_candidates = [prep_dir / 'log.txt', prep_dir / 'processlog.txt']
15  ready_candidates = [prep_dir / 'ready.csv', prep_dir / 'readinesscheck.csv']
16  valid_candidates = [prep_dir / 'valid.csv', prep_dir / 'datasetvalidationreport.csv']
17  zero_candidates = [prep_dir / 'go_zero_label.csv', prep_dir / 'zero_label.csv',
18                    prep_dir / 'gozerolabel.csv']
19
20  def first_existing(candidates):
21      for p in candidates:
22          if p.exists():
23              return p
24      return None
25
26  profile_path = first_existing(profile_candidates)
27  log_path = first_existing(log_candidates)
28  ready_path = first_existing(ready_candidates)
29  valid_path = first_existing(valid_candidates)
30  zero_path = first_existing(zero_candidates)
31
32  checks = pd.DataFrame([
33      {'artifact': 'config', 'path': str(config_path), 'exists': config_path.exists()},
34      {'artifact': 'profile', 'path': str(profile_path) if profile_path else '',
35       'exists': profile_path is not None},

```

```

33     {'artifact': 'log', 'path': str(log_path) if log_path else '', 'exists': log_path
        is not None},
34     {'artifact': 'ready', 'path': str(ready_path) if ready_path else '', 'exists':
        ready_path is not None},
35     {'artifact': 'valid', 'path': str(valid_path) if valid_path else '', 'exists':
        valid_path is not None},
36     {'artifact': 'zero_label', 'path': str(zero_path) if zero_path else '', 'exists':
        zero_path is not None},
37 ])
38 checks
39
40 if not config_path.exists():
41     raise FileNotFoundError(f'Missing config file: {config_path}')
42
43 config = json.loads(config_path.read_text(encoding='utf-8'))
44 paths = config.get('paths', {})
45 cfg = config.get('cfg', {})
46
47 if profile_path is None:
48     raise FileNotFoundError('No profile-like file found. Expected one of: profile.csv,
        corpus_profile_report.csv, readinesscheck.csv')
49
50 profile = pd.read_csv(profile_path)
51 profile.head()
52
53 def safe_columns(df, wanted):
54     return [c for c in wanted if c in df.columns]
55
56 preferred = ['dataset', 'split', 'rows', 'n_rows', 'mean_words', 'median_words',
    'p25_words', 'p75_words', 'min_words', 'max_words']
57 profile_summary = profile[safe_columns(profile, preferred)].copy()
58 if profile_summary.empty:
59     profile_summary = pd.DataFrame({'available_columns': profile.columns.tolist()})
60
61 profile_summary.to_csv(out_dir / 'step5_profile_summary.csv', index=False,
    encoding='utf-8-sig')
62 profile_summary
63
64 label_map_rows = []
65 for key in ['positive', 'negative', 'neutral_other', 'priority']:
66     val = cfg.get(key, [])
67     if isinstance(val, list):
68         val = ', '.join(map(str, val))
69     label_map_rows.append({'key': key, 'value': val})
70 label_map = pd.DataFrame(label_map_rows)
71 label_map.to_csv(out_dir / 'step5_label_mapping.csv', index=False,
    encoding='utf-8-sig')
72 label_map
73
74 log_text = ''
75 if log_path is not None:
76     log_text = log_path.read_text(encoding='utf-8', errors='ignore')
77 log_preview = log_text[:1500]
78 log_summary = pd.DataFrame([
79     'log_found': log_path is not None,
80     'log_path': str(log_path) if log_path else '',
81     'chars_read': len(log_text),
82     'preview': log_preview
83 ])
84 log_summary.to_csv(out_dir / 'step5_log_summary.csv', index=False,
    encoding='utf-8-sig')
85 log_summary
86
87 repro = pd.DataFrame([
88     {'item': 'seed', 'value': cfg.get('seed', 42)},
89     {'item': 'config_path', 'value': str(config_path)},
90     {'item': 'profile_path', 'value': str(profile_path) if profile_path else ''},
91     {'item': 'log_path', 'value': str(log_path) if log_path else ''},

```

```

92     {'item': 'ready_path', 'value': str(ready_path) if ready_path else ''},
93     {'item': 'valid_path', 'value': str(valid_path) if valid_path else ''},
94 ]])
95 repro.to_csv(out_dir / 'step5_reproducibility.csv', index=False, encoding='utf-8-sig')
96 repro
97
98 summary = pd.DataFrame([
99     {'metric': 'profile_exists', 'value': profile_path is not None},
100    {'metric': 'log_exists', 'value': log_path is not None},
101    {'metric': 'ready_exists', 'value': ready_path is not None},
102    {'metric': 'valid_exists', 'value': valid_path is not None},
103    {'metric': 'zero_label_exists', 'value': zero_path is not None},
104    {'metric': 'profile_rows', 'value': len(profile)},
105    {'metric': 'profile_columns', 'value': len(profile.columns)},
106 ])
107 summary.to_csv(out_dir / 'step5_execution_summary.csv', index=False,
108                encoding='utf-8-sig')
109 summary
110
111 md_lines = ['# Step 5 File Inventory', '', 'This notebook reports the key data-prep
112            outputs found in the project.']
113 for _, row in checks.iterrows():
114     md_lines.append(f"- {row['artifact']}: {row['path'] if row['exists'] else
115                    'missing'}")
116 (out_dir / 'step5_inventory.md').write_text('\n'.join(md_lines), encoding='utf-8')
117 '\n'.join(md_lines)

```

Listing 7. `step5_static_report.ipynb`